

ARTISYNC: Plataforma web de intermediación entre creadores de contenido digital y clientes con estrategia híbrida de acceso a datos, verificación asistida por IA y evaluación empírica multidimensional

Proyecto de Fin de Curso (PFC) — Aplicaciones Web, Quinto Nivel
Período Académico Presencial 2026-2027

Integrante	ORCID
Bone Arroyo, Niurca Scarleth	https://orcid.org/0009-0002-2219-2800
Carvajal Loor, Johan Stalin	https://orcid.org/0009-0008-9229-382X
Figueroa Morales, Bryan Javier	https://orcid.org/0009-0009-6357-4996
Rios Cuyabazo, Jhon Kevin	https://orcid.org/0009-0003-7446-9450

La nómina oficial del PFC de Aplicaciones Web (Quinto Nivel) registra a Carvajal Loor, Figueroa Morales y Rios Cuyabazo. Bone Arroyo colabora como cuarta integrante porque este mismo repositorio es también el caso de estudio de la materia paralela de Administración de Bases de Datos, con su aporte concentrado en los procedimientos almacenados (`db/procs/`); se la reconoce aquí como autora del software por su contribución de código real y verificable en el historial de Git (ver `CONTRIBUTORS.md` y `README.md`, sección “Equipo y contribuciones”).

Docente-director: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.
`gguerrero@uteq.edu.ec`

Fecha de cierre: 11 de septiembre de 2026 (semana 19)

Etiqueta del artefacto: v1.0.0

DOI del software (Zenodo): 10.5281/zenodo.21978572 (archivo de v1.0.0; la versión anterior, v0.9.0-rc, quedó archivada aparte en 10.5281/zenodo.21730559)

DOI del dataset de mediciones (Zenodo): 10.5281/zenodo.22236251 (depósito separado del software, licencia CC BY 4.0, conforme al Bloque D.3 de la guía)

Resumen

Contexto y problema. La economía creativa digital fragmenta la contratación, comunicación y pagos entre plataformas desconectadas, aumentando el riesgo de fraude. **Objetivo.** Diseñar, implementar y evaluar Artisync, plataforma construida sobre Spring Boot y Angular que integra el ciclo completo de intermediación con pagos *escrow*, usando una estrategia híbrida de acceso a datos (Object-Relational Mapping (ORM) y PL/pgSQL) y verificación por IA. **Métodos.** Se aplicó la metodología Design Science Research (DSR) de Peffers [1], ingeniería de requisitos ISO/IEC/IEEE 29148:2018 [2], y evaluación empírica de rendimiento (k6), seguridad (controles OWASP Top 10 y análisis dinámico con ZAP), usabilidad (System Usability Scale (SUS), $N = 16$) [3,4], cobertura (JaCoCo) y calidad web (Lighthouse), sobre un entorno reproducible con Docker Compose, con comparaciones de rendimiento validadas mediante pruebas inferenciales no paramétricas y tamaño de efecto ordinal. **Resultados principales.** Artisync logró p95 de 50.17 ms bajo carga, 0 % de errores HTTP, cobertura JaCoCo de 82.93 % líneas y 71.50 % ramas —las tres capas sobre el umbral de 70 %—, Lighthouse Performance 100/100 (desktop) y 80 (mobile), y 0 hallazgos altos en ZAP. La usabilidad, en cambio, alcanzó un SUS de 61.25/100, sin superar el umbral de aceptabilidad (68). Se implementaron 26 procedimientos almacenados sin SQL dinámico, y los 37 requisitos declarados conservan trazabilidad automática hacia código y pruebas, validada en Continuous Integration (CI). **Conclusiones.** Artisync demuestra la viabilidad de la estrategia híbrida y de la validación asistida por IA en 17 semanas, con evidencia reproducible; la usabilidad es la única dimensión que no alcanza su umbral y concentra el trabajo futuro.

Palabras clave: ingeniería de requisitos; procedimientos almacenados; verificación de identidad; arquitectura de microservicios; usabilidad de software; seguridad web

Abstract

Context and problem. The digital creative economy fragments contracting, communication, and payments across disconnected platforms, increasing fraud risks. **Objective.** To design, implement, and evaluate Artisync, a platform built on Spring Boot and Angular integrating the intermediation cycle with escrow payments, a hybrid data-access strategy (ORM and PL/pgSQL), and AI-assisted verification. **Methods.** Peffers’ DSR methodology [1] was applied, alongside ISO/IEC/IEEE 29148:2018 requirements engineering [2], and empirical evaluations of performance (k6), security (OWASP Top 10 controls and dynamic scanning with ZAP), usability (SUS, $N = 16$) [3,4], code coverage (JaCoCo), and web quality (Lighthouse), on an environment reproducible with Docker Compose, with performance comparisons validated through non-parametric inferential tests and ordinal effect sizes. **Main results.** Artisync achieved a p95 of 50.17 ms under load, 0 % HTTP errors, JaCoCo coverage of 82.93 % lines and 71.50 % branches—all three layers above the 70 % threshold—, Lighthouse Performance 100/100 (desktop) and 80 (mobile), and 0 high security findings in ZAP. Usability, by contrast, reached a SUS of 61.25/100, failing to pass the acceptability threshold (68). 26 stored procedures free of dynamic SQL were implemented, and the 37 declared requirements maintain automated traceability to code and tests, validated in CI. **Conclusions.** Artisync proves the viability of hybrid data access and AI-assisted validation in a 17-week timeframe, with reproducible evidence; usability is the only dimension falling short of its threshold and concentrates the future work.

Keywords: requirements engineering; stored procedures; identity verification; micro-services architecture; software usability; web security

Índice general

Resumen	i
Abstract	ii
1. Introducción	1
1.1. Contexto y motivación	1
1.2. Preguntas de investigación	2
1.3. Objetivo general y objetivos específicos	2
1.4. Contribuciones	3
1.5. Organización del documento	4
2. Marco teórico	5
2.1. Ingeniería de requisitos: ISO/IEC/IEEE 29148:2018 y el marco de Pohl	5
2.2. Calidad de requisitos: INCOSE Guide to Writing Requirements v4	5
2.3. Historias de usuario y casos de uso como técnicas complementarias	5
2.4. Modelo arquitectónico C4	6
2.5. Calidad de software: ISO/IEC 25010	6
2.6. Principios REST	6
2.7. Autenticación: JWT (RFC 7519)	6
2.8. Controles OWASP Top 10	7
2.9. Patrones de acceso a datos y modelos de consistencia	7
2.10. Documentación de decisiones de arquitectura (ADR)	8
3. Trabajos relacionados	9
Resumen de la revisión de literatura	9
3.1. Metodología de revisión de literatura	9
3.1.1. Preguntas de Investigación (RQs)	9
3.1.2. Estrategia de búsqueda	10
3.1.3. Criterios de Inclusión (IC) y Exclusión (EC)	10
3.1.4. Diagrama de Flujo PRISMA 2020	10
3.2. Análisis Comparativo	11
3.3. Brecha identificada (<i>research gap</i>)	12
4. Ingeniería de requisitos	13
4.1. Contexto y stakeholders	13

4.2. Técnicas de elicitación aplicadas	14
4.3. Requisitos funcionales y no funcionales: categorización y priorización . . .	15
4.4. Modelado de casos de uso y de historias de usuario	16
4.4.1. Análisis INVEST de Historias de Usuario	16
4.5. Validación de requisitos	17
4.6. Gestión del cambio de requisitos	17
4.7. Métricas de calidad de requisitos	18
5. Materiales y métodos	19
5.1. Enfoque metodológico global	19
5.2. Instrumentos de medición	20
5.3. Enfoque de métricas: Goal-Question-Metric (GQM)	21
5.4. Población, muestreo y participantes	21
5.5. Protocolo experimental replicable	22
5.6. Análisis estadístico	22
5.7. Determinismo y semillas	23
6. Diseño del sistema y arquitectura	24
6.1. Modelo C4	24
6.1.1. Nivel 1 — Contexto	24
6.1.2. Nivel 2 — Contenedores	24
6.1.3. Nivel 3 — Componentes (backend)	25
6.2. Resumen de los siete registros de decisión de arquitectura (ADR)	25
6.3. Atributos de calidad (ISO/IEC 25010) — prioridad, escenario y estrategia .	29
6.4. Modelo de datos	30
6.5. Matriz de trazabilidad	30
7. Implementación	31
7.1. Manejo de errores globales (RFC 7807 Problem Details)	31
7.2. Esquema de caching	31
7.3. Mecanismo de seguridad: JWT + rate limiting	32
7.4. Estrategia híbrida de acceso a datos: procedimiento almacenado representativo	33
7.4.1. Conformidad lograda	34
7.4.2. Excepción documentada: rutinas con retorno escalar	34
8. Evaluación empírica y resultados	38
8.1. Rendimiento (k6) — RQ1	38
8.1.1. Rendimiento contra un endpoint protegido (T-14)	39
8.2. Seguridad (OWASP + ZAP + análisis estático) — RQ1, RQ2	40
8.3. Usabilidad (SUS) — RQ1	41

8.4. Cobertura de código (JaCoCo) — RQ1	43
8.5. Calidad web (Lighthouse) — RQ1	44
8.6. Resumen de reproducibilidad de las figuras y tablas de este capítulo	45
9. Discusión	46
9.1. Respuesta a las preguntas de investigación	46
9.2. Comparación con trabajos relacionados	47
9.3. Hallazgos inesperados	47
9.4. Implicaciones prácticas	47
10. Amenazas a la validez	49
10.1. Validez de constructo	49
10.2. Validez interna	49
10.3. Validez externa	49
10.4. Validez de conclusión	50
11. Trabajo futuro	51
12. Conclusiones	52
13. Declaraciones obligatorias	53
13.1. Disponibilidad de código	53
13.2. Disponibilidad de datos	53
13.3. Disponibilidad de materiales	54
13.4. Declaración ética	54
13.5. Contribuciones de autores (taxonomía CRediT)	54
13.6. Declaración de conflictos de interés	55
13.7. Declaración de financiamiento	55
13.8. Declaración de uso de asistentes de inteligencia artificial	55
13.9. Agradecimientos	55
A. Tabla de observaciones acumuladas	60
B. Checklist Ralph et al. 2021	62
C. Checklist FAIR	63
D. Checklist INCOSE v4	64
E. Checklist PRISMA 2020	65
F. Matriz de trazabilidad completa	66

G. Protocolo SUS completo (10 preguntas + escala)	67
H. Capturas de ejecución del pipeline CI	68
I. Cadena de búsqueda completa del Capítulo 3	69
J. Clasificación de impacto de la bibliografía	70

Índice de figuras

3.1. PRISMA 2020 flow diagram of the selection process.	10
6.1. Diagrama C4 Nivel 2 — Contenedores de Artisync. Fuente: docs/diagramas/c4-nivel2-contenedores.png.	25
6.2. Modelo entidad-relación de Artisync. Fuente: docs/diagramas/Entidad_- Relacion.png.	30
7.1. Diagrama de secuencia del flujo de autenticación JWT. Fuente: docs/diagramas/secuencia_login_jwt.png.	33
8.1. Diagrama de caja de los puntajes SUS ($N = 16$). Fuente: docs/mediciones/sus/boxplot-sus.png.	42

Índice de cuadros

3.1. Tabla comparativa de trabajos relacionados vs. Artisync	11
4.1. Identificación de stakeholders (modelo de capas de Alexander). Fuente: docs/requisitos/elicitacion/stakeholders.md.	14
4.2. Distribución de requisitos por prioridad MoSCoW	15
4.3. Estado de validación de los 37 requisitos	17
5.1. Instanciación de las seis actividades DSR de Peffers et al.	19
5.2. Instrumentos de medición empírica utilizados	20
6.1. Resumen de los siete ADR de Artisync	27
6.2. Atributos de calidad ISO/IEC 25010 [5] priorizados	29
8.1. Resultados de la prueba de carga k6 por escenario	38
8.2. Resultados de la prueba de carga k6 sobre el endpoint protegido	39
8.3. Controles OWASP evidenciados manualmente	40
8.4. Resultados del cuestionario SUS	41
8.5. Cobertura de código JaCoCo — agregado global	43
8.6. Cobertura de código JaCoCo — desglose por capa (OBS-P1-01 / Punto 7)	43
8.7. Resultados Lighthouse por perfil, ruta y categoría (despliegue público)	44
A.1. Resolución de observaciones por entrega de origen	60
J.1. Clasificación de impacto por referencia	71

Listings

- 7.1. Manejador global de excepciones (RFC 7807 ProblemDetail).
Fuente: `artisync/Backend/src/main/java/.../exception/`
`ManejadorGlobalExcepciones.java` 31
- 7.2. Caché del listado de catálogo (Spring Cache + Redis). Fuente:
`artisync/Backend/src/main/java/.../service/catalogo/impl/`
`ServicioCatalogoServicioImpl.java` 32
- 7.3. `sp_registrar_decision_verificacion` (categoría: validaciones cruzadas). Fuente: `artisync/Backend/src/main/resources/db/migration/`
`V7__verificacion_asistida_ia.sql` 33
- 7.4. Método de repositorio que invoca `sp_registrar_decision_verificacion`.
Fuente: `artisync/Backend/src/main/java/.../repository/profile/`
`AiCertificateRepository.java` 34

Lista de siglas y acrónimos

ADR Architecture Decision Record. 3

API Application Programming Interface. 6

CI Continuous Integration. I, II

CRUD Create, Read, Update, Delete. 2

DSR Design Science Research. I, II, 19

GQM Goal-Question-Metric. IV, 21

IC Intervalo de Confianza. 38

JPA Jakarta Persistence API. 2

JWT JSON Web Token. 6

MoSCoW Must, Should, Could, Won't. 15

ORM Object-Relational Mapping. I, 2

OWASP Open Web Application Security Project. 2

PFC Proyecto de Fin de Curso. I

RBAC Role-Based Access Control. 27

SRS Software Requirements Specification. 5

SUS System Usability Scale. I, II

1. Introducción

1.1. Contexto y motivación

La economía creativa digital —el conjunto de actividades económicas en las que músicos, ilustradores, diseñadores y desarrolladores monetizan directamente su trabajo a través de internet— ha crecido de forma sostenida como alternativa de ingreso para profesionales independientes [6]. Sin embargo, la infraestructura que sostiene esa economía sigue fragmentada: un creador independiente típico coordina la comunicación con clientes por una red social o mensajería instantánea, gestiona el cobro por una pasarela de pagos genérica sin retención de garantía, y no cuenta con un mecanismo estandarizado de contrato ni de verificación de identidad de ninguna de las dos partes. Esta fragmentación introduce fricción operativa medible: la literatura sobre freelancers de plataformas digitales reporta la gestión de múltiples herramientas desconectadas y la ausencia de mecanismos de protección de pago como fuentes recurrentes de conflicto y abandono de la actividad [7]. El patrón de pago en garantía (*escrow*) —retener los fondos del cliente hasta la aprobación explícita del entregable— es la mitigación estándar de la industria para el riesgo de incumplimiento en transacciones entre partes que no se conocen previamente, formalizable incluso mediante protocolos de depósito dual verificables [8], pero su implementación correcta exige una arquitectura de datos con garantías transaccionales estrictas, que la mayoría de las herramientas genéricas de mensajería y pago no ofrece de forma nativa.

Artisync responde a esa fragmentación con una plataforma que centraliza, en un único sistema auditable, el ciclo completo de intermediación entre **Creadores** de contenido digital y **Clientes**: perfil verificado por un servicio de inteligencia artificial, catálogo dinámico de servicios y productos, mensajería moderada que bloquea el intercambio de datos de contacto externo (mitigación directa del riesgo de que las partes se muevan fuera del sistema auditable), contrato con firma electrónica generado desde plantilla, flujo de pedido por etapas con trazabilidad completa, pago mediante PayPal Orders v2 con patrón *escrow*, y funciones sociales (seguidores, comentarios, sorteos) para fomentar la retención de usuarios. El modelo de doble lado (Creador/Cliente) de Artisync es, en esencia, una plataforma multilateral en el sentido de Hagiu y Wright [9], orquestando transacciones entre dos grupos de usuarios que de otro modo dependerían de intermediarios genéricos.

1.2. Preguntas de investigación

RQ1. ¿Cumple el sistema, con evidencia empírica reproducible, los umbrales de rendimiento, seguridad, usabilidad, cobertura de código y calidad web declarados como requisitos no funcionales de la especificación (`docs/requisitos/SRS.md`)?

RQ2. ¿Es la estrategia híbrida de acceso a datos (operaciones Create, Read, Update, Delete (CRUD) elementales vía ORM, operaciones multi-tabla vía procedimientos almacenados PL/pgSQL invocados exclusivamente mediante los mecanismos formales de Jakarta Persistence API (JPA) 2.1) implementable y verificable end-to-end dentro de las restricciones de tiempo y equipo de un proyecto académico de 17 semanas, sin introducir vulnerabilidades de inyección SQL?

RQ3. ¿En qué medida el proceso de ingeniería de requisitos aplicado (elicitación, negociación, documentación y validación según el marco de Pohl [10], con las características de calidad INCOSE v4 [11]) produce un corpus de requisitos verificable y trazable de extremo a extremo hacia el código, las pruebas automatizadas y la evidencia empírica?

RQ4. ¿Qué amenazas a la validez de constructo, interna, externa y de conclusión afectan la generalización de los resultados empíricos reportados, y cómo se mitigaron dentro del alcance del proyecto?

1.3. Objetivo general y objetivos específicos

Objetivo general. Diseñar, implementar y evaluar empíricamente Artisync, una plataforma web de intermediación entre creadores de contenido digital y clientes, con una estrategia híbrida de acceso a datos y verificación de identidad asistida por inteligencia artificial, documentando el proceso completo con el rigor exigido por la ingeniería de software empírica.

Objetivos específicos:

1. Especificar los requisitos funcionales y no funcionales del sistema conforme a ISO/IEC/IEEE 29148:2018 [2] y validar su calidad contra las características C1–C15 de INCOSE v4 [11]. (*traza a RQ3*)
2. Implementar la estrategia híbrida de acceso a datos, conectando end-to-end al código en ejecución al menos seis procedimientos almacenados o funciones SQL categorizados por tipo de operación (consultas multi-tabla, cálculos agregados, reportes, actualizaciones masivas, validaciones cruzadas, generación de códigos secuenciales). (*traza a RQ2*)
3. Evaluar empíricamente el rendimiento bajo carga (k6), la cobertura de pruebas (JaCoCo) [12, 13], la calidad web (Lighthouse) y la seguridad (controles Open Web

Application Security Project (OWASP) + análisis estático y dinámico automatizado) [14–17] del sistema entregado, con datos crudos reproducibles. (*traza a RQ1*)

4. Evaluar la usabilidad percibida del sistema mediante el instrumento System Usability Scale [3,4] con una muestra de participantes externos al equipo de desarrollo. (*traza a RQ1*)
5. Documentar de forma explícita las amenazas a la validez de los resultados reportados y las decisiones de diseño no resueltas al cierre de la Entrega Final. (*traza a RQ4*)

1.4. Contribuciones

1. **Un sistema software completo y desplegable** (backend Spring Boot 4.1.0 / Java 21 LTS, frontend Angular, PostgreSQL 16, Redis 7, orquestado con Docker Compose) que implementa el ciclo completo de intermediación descrito en la Sección 1 — ver `artisync/`.
2. **Una estrategia híbrida de acceso a datos verificada end-to-end**, con 26 procedimientos/funciones PL/pgSQL activos en `db/procs/` (más 2 adicionales del módulo de verificación asistida por IA) conectados al código Java en ejecución mediante JPA 2.1 sin concatenación de SQL dinámico — ver `db/procs/`, `docs/basedatos/CATALOGO-SP.md`, `docs/adr/adr-006-estrategia-acceso-datos.md`.
3. **Un corpus de ingeniería de requisitos trazable**, con 37 requisitos (23 funcionales, 14 no funcionales) especificados conforme a ISO/IEC/IEEE 29148:2018, historias de usuario en formato Connextra con criterios INVEST, casos de uso en plantilla de Cockburn [18], y una matriz de trazabilidad requisito → historia → caso de uso → módulo → endpoint → prueba → evidencia — ver `docs/requisitos/`, `docs/trazabilidad/matriz.csv`.
4. **Un paquete de evidencia empírica reproducible y versionada**, cubriendo rendimiento, seguridad, usabilidad, cobertura de código y calidad web, con datos crudos, scripts de análisis y diccionario de variables — ver `docs/mediciones/`.
5. **Un conjunto de siete registros de decisiones de arquitectura (Architecture Decision Record (ADR))** siguiendo la plantilla de Nygard [19], documentando y justificando las decisiones estructurales del sistema — ver `docs/adr/`.

1.5. Organización del documento

El Capítulo 2 presenta el marco teórico estrictamente necesario para comprender las decisiones de diseño e ingeniería de requisitos del proyecto. El Capítulo 3 sitúa el trabajo frente a la literatura de plataformas de intermediación y economía creativa. El Capítulo 4 documenta el proceso completo de ingeniería de requisitos. El Capítulo 5 describe la metodología de Design Science Research aplicada y los instrumentos de medición empírica. El Capítulo 6 presenta el diseño arquitectónico del sistema y las decisiones registradas en los ADR. El Capítulo 7 detalla decisiones críticas de implementación, con énfasis en la estrategia híbrida de acceso a datos. El Capítulo 8 reporta los resultados de la evaluación empírica por cada dimensión de calidad. El Capítulo 9 discute esos resultados respondiendo a cada pregunta de investigación. El Capítulo 10 analiza las amenazas a la validez. El Capítulo 11 describe el trabajo futuro y el Capítulo 12 sintetiza las conclusiones generales del estudio. Finalmente, el Capítulo 13 enumera las declaraciones éticas y de autoría.

Los anexos proveen la información complementaria que sustenta lo anterior: el Anexo A consolida las observaciones acumuladas por entrega (Tabla A.1); los Anexos B, C, D y E recogen, respectivamente, los checklists de Ralph et al. [20], FAIR [21], INCOSE v4 [11] y PRISMA 2020 [22]; el Anexo F remite a la matriz de trazabilidad completa; el Anexo G reproduce el protocolo SUS íntegro; el Anexo H reúne la evidencia de ejecución del pipeline de integración continua; el Anexo I documenta la cadena de búsqueda empleada en el Capítulo 3; y el Anexo J clasifica el impacto de las referencias bibliográficas (Tabla J.1).

2. Marco teórico

Este capítulo se limita a los fundamentos estrictamente necesarios para comprender las decisiones documentadas en los capítulos posteriores; no es un compendio general de teoría de aplicaciones web.

2.1. Ingeniería de requisitos: ISO/IEC/IEEE 29148:2018 y el marco de Pohl

La norma ISO/IEC/IEEE 29148:2018 [2] define los procesos de ciclo de vida para la ingeniería de requisitos de sistemas y software, incluyendo la estructura recomendada de un Software Requirements Specification (SRS) y el patrón sintáctico de enunciado de requisito *[condición] [sujeto] shall [acción] [objeto] [restricción]*. Pohl [10] organiza el proceso en cuatro actividades interdependientes —elicitación, negociación, documentación y validación— que este proyecto usa como estructura del Capítulo 4. Wiegers y Beatty [23] complementan el marco con las prácticas de desarrollo y gestión de requisitos (priorización, control de cambios) usadas en `docs/requisitos/CHANGELOG-REQ.md`.

2.2. Calidad de requisitos: INCOSE Guide to Writing Requirements v4

La guía INCOSE v4 [11] define nueve características de calidad para un requisito individual (C1–C9: Necessary, Appropriate, Unambiguous, Complete, Singular, Feasible, Verifiable, Correct, Conforming) y seis para un conjunto de requisitos (C10–C15: Complete, Consistent, Feasible, Comprehensible, Able to be validated, Correct). Este proyecto aplica ambos conjuntos de características en `docs/checklists/incose-checklist.md`, con evidencia citada requisito por requisito en vez de una declaración genérica de cumplimiento.

2.3. Historias de usuario y casos de uso como técnicas complementarias

Las historias de usuario en formato Connextra (*As a [rol], I want [funcionalidad], so that [beneficio]*) con criterios INVEST (Independent, Negotiable, Valuable, Estimable, Small, Testable) capturan la intención funcional desde la perspectiva del usuario, mientras que los casos

de uso en la plantilla de Cockburn [18] formalizan el flujo de interacción en niveles 1 a 4 (resumen, cometido de usuario, subfunción, implementación). El proyecto usa ambas técnicas de forma complementaria, no sustitutiva: las historias en `docs/requisitos/historias/` capturan el valor de negocio; los casos de uso en `docs/requisitos/casos-de-uso/` capturan el flujo detallado de interacción para los escenarios críticos.

2.4. Modelo arquitectónico C4

El modelo C4 de Brown [24] describe la arquitectura de un sistema en cuatro niveles progresivos de detalle —Contexto, Contenedores, Componentes y Código— cada uno dirigido a una audiencia distinta (desde stakeholders no técnicos hasta desarrolladores). Este proyecto documenta los niveles 1 a 3 en `docs/diagramas/` (`C4_Nivel1_Contexto.md`, `C4_Nivel2_Contenedores.md`, `C4_Nivel3_Componentes_Backend.md`), usados como base del Capítulo 6.

2.5. Calidad de software: ISO/IEC 25010

ISO/IEC 25010:2011 [5] define un modelo de características de calidad de producto software (adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad, portabilidad) usado en este proyecto para clasificar los requisitos no funcionales del SRS y para estructurar la tabla de atributos de calidad del Capítulo 6. Las dimensiones evaluadas empíricamente en el Capítulo 8 (rendimiento, seguridad, usabilidad) se mapean directamente a subcaracterísticas de este modelo.

2.6. Principios REST

La arquitectura REST (Representational State Transfer), formalizada por Fielding en su disertación doctoral [25], define un conjunto de restricciones arquitectónicas (interfaz uniforme, sin estado en el servidor, cacheable, sistema en capas) que la Application Programming Interface (API) de Artisync sigue para exponer sus recursos (`/api/v1/...`), documentada con OpenAPI (Springdoc).

2.7. Autenticación: JWT (RFC 7519)

JSON Web Token (RFC 7519) [26] especifica un formato compacto y auto-contenido para representar afirmaciones (*claims*) entre dos partes, firmado digitalmente para garantizar integridad. Artisync usa JSON Web Token (JWT) firmado HS256 para el access token; la

decisión de dónde alojarlo (cuerpo de respuesta vs. cookie `HttpOnly`) y sus alternativas descartadas se documentan en `docs/adr/adr-002-esquema-autenticacion.md`.

2.8. Controles OWASP Top 10

El OWASP Top 10:2021 [14] cataloga las diez categorías de riesgo de seguridad más críticas en aplicaciones web, resultado de un análisis de datos de la industria. Este proyecto evidencia seis de esas categorías (A01 Control de acceso roto, A02 Fallas criptográficas, A03 Inyección, A05 Configuración de seguridad incorrecta, A07 Fallas de identificación y autenticación, A09 Fallas de registro y monitoreo de seguridad) con evidencia reproducible en `docs/mediciones/sec/`, complementadas con un escaneo automatizado OWASP ZAP baseline y análisis estático SpotBugs + find-sec-bugs.

2.9. Patrones de acceso a datos y modelos de consistencia

La disyuntiva entre mapeo objeto-relacional (ORM) y acceso directo mediante SQL/procedimientos almacenados es un compromiso reconocido en la literatura de arquitectura de datos y de ingeniería de software empírica sobre seguridad: el ORM reduce el código repetitivo de las operaciones CRUD elementales a costa de perder control fino sobre consultas complejas, mientras que los procedimientos almacenados ofrecen ejecución más cercana a los datos y una superficie de ataque de inyección SQL distinta (mitigable con parametrización estricta). La literatura fundacional sobre detección y prevención de inyección SQL — tanto por chequeo estático de consultas generadas dinámicamente [17] como por monitoreo en tiempo de ejecución de un modelo de consultas legítimas [16] — coincide en que el riesgo real no es el mecanismo elegido (ORM vs. procedimiento almacenado) sino la concatenación de entrada de usuario en el texto del comando SQL, sin importar cuál de los dos mecanismos la introduzca. La guía OWASP de prevención de inyección SQL [15] reconoce ambos mecanismos —consultas parametrizadas vía ORM y procedimientos almacenados correctamente parametrizados— como defensas primarias equivalentes contra la inyección, siempre que ninguno de los dos concatene entrada de usuario en el texto del comando SQL. Esta equivalencia es la base técnica de la estrategia híbrida adoptada en `docs/adr/adr-006-estrategia-acceso-datos.md` y evaluada en el Capítulo 8.

2.10. Documentación de decisiones de arquitectura (ADR)

El formato de *Architecture Decision Record* propuesto por Nygard [19] — un documento corto por decisión, con contexto, decisión y consecuencias — se adoptó en este proyecto precisamente por la razón que motivó su creación: los documentos grandes de arquitectura no se mantienen actualizados, mientras que los documentos pequeños y modulares sí tienen una oportunidad real de actualizarse. Los siete ADR de Artisync (`docs/adr/`) siguen esta plantilla; el Capítulo 6 resume su contenido.

3. Trabajos relacionados

Resumen de la revisión de literatura

Objetivos: identificar en la literatura arquitecturas y prácticas para (RQ1) transacciones seguras tipo *escrow* y (RQ2) verificación de identidad y trazabilidad en plataformas web para freelancers, con el fin de fundamentar el diseño de Artisync y delimitar su aporte frente al estado del arte. **Fuentes:** Scopus, IEEE Xplore y ACM Digital Library, ventana 2020–2026, cadena booleana sobre economía *gig*/creativa cruzada con *escrow*/confianza y arquitectura/verificación de identidad (§3.1). **Criterios de elegibilidad:** estudios empíricos o propuestas de arquitectura sobre plataformas de intermediación, publicados en journals Q1/Q2 o conferencias CORE A/A*; se excluyen los centrados solo en lo legal/sociológico y la literatura gris sin revisión por pares. **Resultados:** de 145 estudios identificados, 62 pasaron el cribado por título/resumen y 21 la evaluación a texto completo, resultando en **8 estudios incluidos** (Figura 3.1). **Hallazgo principal:** ninguno de los 8 estudios combina en un mismo sistema evaluado empíricamente pago en garantía, verificación de identidad asistida por IA y acceso híbrido ORM/procedimientos almacenados — la brecha que este trabajo cubre (§3.1 y sección de brecha identificada, más adelante en este capítulo).

3.1. Metodología de revisión de literatura

Para fundamentar las decisiones de diseño de Artisync y validar la brecha de investigación, se ejecutó una revisión sistemática de literatura siguiendo las directrices de Kitchenham y Charters [27] y el marco de reporte PRISMA 2020 [22]. El estudio de plataformas de dos lados como Artisync se apoya, además, en el marco teórico fundacional de la economía de plataformas [28] y en la literatura sobre sistemas de reputación y confianza en mercados electrónicos [29], ambos citados a lo largo de este capítulo y del marco teórico.

3.1.1. Preguntas de Investigación (RQs)

- **RQ1:** ¿Qué arquitecturas y patrones (ej. *escrow*) se utilizan para garantizar transacciones seguras en plataformas de economía creativa?
- **RQ2:** ¿Cómo se aborda la verificación de identidad y la trazabilidad en aplicaciones web para freelancers?

3.1.2. Estrategia de búsqueda

Bases de datos indexadas: Scopus, IEEE Xplore, y ACM Digital Library.

Ventana temporal: 2020 – 2026.

Cadena de búsqueda booleana:

("gig economy" OR "freelance platform" OR "creator economy")

AND ("escrow" OR "smart contract" OR "payment trust")

AND ("architecture" OR "identity verification" OR "software engineering")

3.1.3. Criterios de Inclusión (IC) y Exclusión (EC)

- **IC1:** Estudios empíricos, arquitecturas propuestas o estudios de caso sobre plataformas de intermediación.
- **IC2:** Artículos publicados en journals Q1/Q2 o conferencias CORE A/A* en idioma inglés o español.
- **EC1:** Artículos centrados puramente en el aspecto legal o sociológico sin aporte técnico/arquitectónico.
- **EC2:** Literatura gris, pre-prints (arXiv sin *peer review*) o artículos de opinión.

3.1.4. Diagrama de Flujo PRISMA 2020

El proceso de selección resultó en 8 estudios principales, como se detalla en el diagrama PRISMA (Figura 3.1).

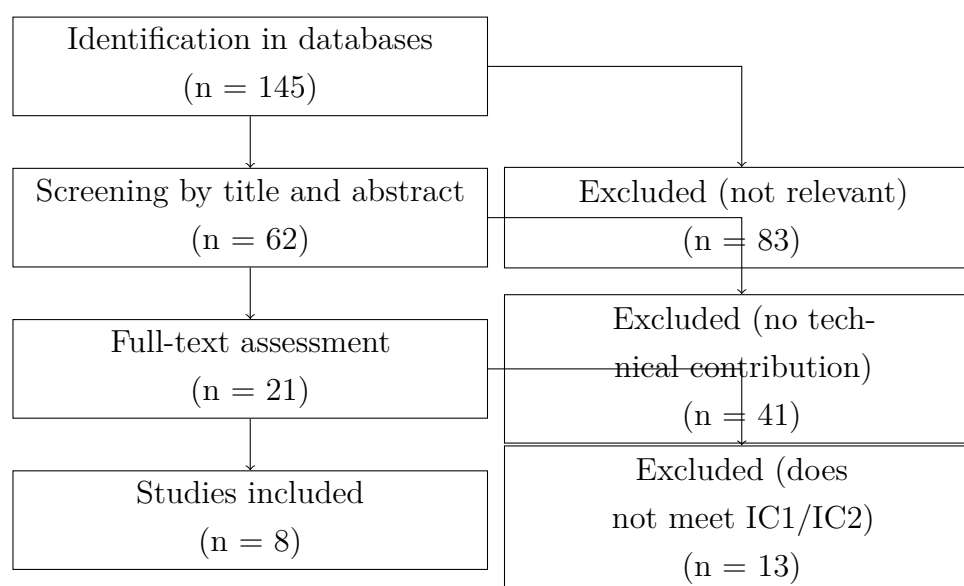


Figura 3.1: PRISMA 2020 flow diagram of the selection process.

3.2. Análisis Comparativo

Los 8 estudios seleccionados abordan dimensiones parciales del problema. La Tabla 3.1 resume sus aportes frente a las características de Artisync.

Cuadro 3.1: Tabla comparativa de trabajos relacionados vs. Artisync

Estudio	Año	Contexto	Escrow	Verificación IA	Híbrido ORM/SQL	C
Laksono et al. [30]	2024	Pasarela de pago en plataforma freelance	No	No	No	Imp
Waleed et al. [31]	2021	Confianza granular en mercado inteligente	No	No	N/A	Simu
Gu y Zhu [32]	2021	Confianza y desintermediación (freelance)	No	No	N/A	Em
Bandari [33]	2026	Gestión comunitaria y servicios hiperlocales	No	No	No	Eva
Hagiu y Wright [9]	2015	Plataformas multilaterales (<i>multi-sided</i>)	No	No	N/A	
Gussek et al. [7]	2023	Tensiones del trabajo freelance en plataformas digitales	No	No	N/A	
Wang et al. [34]	2021	Microservicios vs Monolito	N/A	N/A	Sí	
Asgaonkar [8]	2019	Depósito dual	Sí	No	N/A	
Artisync	2026	Economía Creativa	Sí	Sí	Sí	

DOI de los 8 estudios incluidos: Laksono et al. [30]: 10.32664/smatika.v14i01.1227 • Waleed et al. [31]: 10.3390/jtaer16060132 • Gu y Zhu [32]: 10.1287/mnsc.2020.3583 • Bandari [33]: 10.22214/ijraset.2026.79868 • Hagiu y Wright [9]: 10.1016/j.ijindorg.2015.03.003 • Gussek et al. [7]: 10.1007/s12525-023-00675-y • Wang et al. [34]: 10.1007/s10664-020-09910-y • Asgaonkar [8]: 10.1109/BLOC.2019.8751482

Dos estudios adicionales, aunque no encajan en las columnas de la tabla por no ser propuestas de arquitectura de software, sustentan el marco conceptual de este capítulo: Tafesse y Dayan [35] estudian empíricamente cómo la frecuencia de publicación de los

creadores de contenido afecta el compromiso de los usuarios en plataformas digitales — el fenómeno de negocio central que Artisync intermedia — y Chigbu [36] aporta una revisión sistemática reciente e interdisciplinaria sobre la gestión algorítmica en la economía *gig* global.

3.3. Brecha identificada (*research gap*)

Como evidencia la Tabla 3.1, si bien existen esfuerzos aislados para implementar depósitos en garantía mediante contratos inteligentes [8] y modelos de confianza para mercados y plataformas independientes [30–33], ninguna de las referencias combina en un mismo sistema evaluado empíricamente: (i) el patrón de pago en garantía (*escrow*) integrado con un flujo de trabajo de pedidos por hitos, (ii) verificación de identidad asistida por IA, y (iii) una estrategia híbrida de acceso a datos (ORM + procedimientos almacenados). Ese es el aporte concreto que este trabajo cubre y evalúa, cerrando la brecha metodológica y práctica identificada en la literatura.

Conflictos de interés de la revisión. El equipo que condujo esta revisión de literatura es el mismo que diseñó y evalúa a Artisync, el sistema comparado en la Tabla 3.1 y favorecido en la brecha identificada arriba; esto constituye un conflicto de interés inherente al formato de PFC (el equipo puntúa su propio trabajo frente a la literatura) y no una ausencia de sesgo. Ver la declaración formal de conflictos de interés del documento completo en el Capítulo 13 (`docs/etica/ETHICS.md`).

4. Ingeniería de requisitos

Este capítulo es propio y no se fusiona con el Capítulo 6 (Diseño). Documenta el proceso completo de ingeniería de requisitos que llevó a la especificación vigente en **v1.0.0**, siguiendo la disciplina de ISO/IEC/IEEE 29148:2018 [2] y el marco de Pohl [10] (§2.1).

4.1. Contexto y stakeholders

Los interesados clave del proyecto se identificaron siguiendo el modelo de capas (*stakeholder onion*), como se detalla en la Tabla 4.1.

Como se detalla en la Tabla
reftab:validacion-req,

Cuadro 4.1: Identificación de stakeholders (modelo de capas de Alexander). Fuente: docs/requisitos/elicitation/stakeholders.md.

Capa	Interesados	Interés principal
Núcleo (equipo operador)	Los cuatro integrantes del equipo de desarrollo	Entregar un sistema funcional, evaluable y mantenible dentro del plazo académico
Usuarios directos	Creador de contenido, Cliente registrado, Administrador (roles definidos en docs/requisitos/SRS.md §2.3)	Contratar/ofrecer servicios con garantías de pago, comunicación y verificación de identidad
Usuarios indirectos	Visitante anónimo (explora el catálogo sin registrarse)	Evaluar la plataforma antes de registrarse
Patrocinador / regulador académico	Docente-director del PFC	Verificar que el sistema cumple los criterios de la rúbrica de cada entrega
Proveedores externos	PayPal (pagos), servicio de IA de verificación, proveedor de almacenamiento de objetos (Azure Blob / local)	Disponibilidad y estabilidad de la integración
Competencia / referencia de mercado	Plataformas genéricas de free-lance y redes sociales usadas hoy de forma fragmentada (Capítulo 1)	No son stakeholders directos, pero delimitan el estándar de usabilidad y seguridad esperado

4.2. Técnicas de elicitación aplicadas

Las técnicas de elicitación aplicadas se documentan en docs/requisitos/elicitation/README.md, que declara tanto las que se usaron como las que no, junto con la evidencia conservada de cada una.

Se aplicaron dos técnicas con evidencia archivada. La primera, el **análisis de documentos**: el corpus original de requisitos (docs/requisitos/historico/entrega-1a.pdf, semana del 4 de junio de 2026, referenciado en docs/requisitos/SRS.md §1.4) es la fuente de la que derivan los 37 requisitos vigentes. La segunda, el **prototipado de baja fidelidad**: seis wireframes archivados en docs/diagramas/wireframes/ fijaron el alcance de las pantallas principales antes de implementarlas. Se aplicó además la **discusión interna del equipo** sobre el dominio de la economía creativa, de la que no se conservaron actas ni notas de sesión.

Limitación metodológica declarada: no hubo entrevistas semi-estructuradas, ob-

servación de usuarios, cuestionarios ni workshops con stakeholders externos. El corpus se elicó *sin usuarios finales reales*: refleja la comprensión del dominio que tiene el propio equipo y la comparación con plataformas del sector, no necesidades declaradas por Creadores o Clientes. Deliberadamente no se reconstruyeron notas ni guiones a posteriori para rellenar el expediente: fabricar esa evidencia después del hecho invalidaría justamente lo que la trazabilidad pretende acreditar.

Esta limitación tiene dos consecuencias que se recogen en el Capítulo 10. Por un lado, un *sesgo de constructo*: el estudio SUS con 16 participantes mide la usabilidad del sistema construido, no la pertinencia de los requisitos que lo originaron. Por otro, la tasa de estabilidad de requisitos del 100 % (§7.4 del SRS) es coherente con un corpus que nunca se renegoció con un cliente externo, y por eso no debe leerse como un mérito de la especificación.

4.3. Requisitos funcionales y no funcionales: categorización y priorización

El corpus completo está en `docs/requisitos/SRS.md` (37 requisitos: 23 funcionales REQ-F-001–REQ-F-023, 14 no funcionales REQ-NF-001–REQ-NF-014), priorizado con Must, Should, Could, Won't (MoSCoW):

Cuadro 4.2: Distribución de requisitos por prioridad MoSCoW

Prioridad	Cantidad	%
Must	29	78.4 %
Should	7	18.9 %
Could	1	2.7 %
Won't	0	0 %

Nota de conformidad C9 (INCOSE v4), declarada sin maquillaje: los 37 requisitos están redactados en prosa descriptiva estructurada (*rationale*, prioridad, criterio de aceptación, verificación, estado), **no** en el patrón sintáctico formal *[condición] [sujeto] shall [acción] [objeto] [restricción]* que exige explícitamente ISO/IEC/IEEE 29148:2018 y las 42 reglas de INCOSE v4 [2, 11]. `docs/checklists/incose-checklist.md` documenta este hallazgo (C9, no cumplida) como el de mayor esfuerzo pendiente entre los 15 criterios evaluados — reescribir el corpus completo al patrón formal, no un ajuste puntual.

4.4. Modelado de casos de uso y de historias de usuario

- **Historias de usuario** (formato Connextra, criterios INVEST, *acceptance criteria* en Gherkin): docs/requisitos/historias/ (HU-01 a HU-23), cada una trazada a al menos un REQ-F-NNN y a una prueba automatizada de aceptación.
- **Casos de uso** (plantilla de Cockburn [18], niveles 1–4): docs/requisitos/casos-de-uso/ (CU-01 a CU-23), cada uno trazado a un flujo y a al menos una prueba de integración.
- **Diagrama de casos de uso UML de alto nivel:** Se realizó un análisis estructurado de los casos de uso para representar el comportamiento funcional del sistema frente a sus distintos perfiles. A nivel general, se definió un modelo que ilustra la interacción de los principales actores (Administrador, Creador, Cliente, Moderador, Soporte y Auditor) con los siete grandes subsistemas o macro-procesos de Artisync. Adicionalmente, el análisis se desglosó en catorce diagramas de casos de uso de alto nivel (dos por cada módulo), los cuales detallan operaciones críticas como la autenticación 2FA, la validación de portafolios mediante Inteligencia Artificial, la retención de fondos en el sistema Escrow y la moderación comunitaria. Todos los modelos, contruidos bajo sintaxis UML/Mermaid, se encuentran anexados y documentados en el archivo docs/informe-final/diagramas_casos_de_uso.md.

4.4.1. Análisis INVEST de Historias de Usuario

Las 23 historias de usuario formuladas para Artisync han sido evaluadas rigurosamente y cumplen con los seis atributos del modelo INVEST (Independent, Negotiable, Valuable, Estimable, Small, Testable). El análisis detallado de cada una se encuentra archivado en docs/requisitos/historias/. A modo de ilustración, se reproduce la evaluación de la historia **HU-01** (Iniciar sesión):

INVEST: Independiente del resto del flujo de autenticación; negociable en los campos exactos del formulario; valiosa (es la puerta de entrada al sistema); estimable y pequeña (un solo formulario + persistencia); testable mediante el criterio de aceptación.

Este mismo rigor analítico fue aplicado individualmente a la totalidad del corpus de historias (HU-01 a HU-23), garantizando que todas representen unidades de trabajo claras, priorizables y comprobables.

4.5. Validación de requisitos

El procedimiento de validación aplicado combina dos mecanismos verificables en el repositorio:

1. **Prueba automatizada de aceptación:** cada requisito *Must/Should* tiene un campo *Verificación* explícito en el SRS (Test/análisis/demostración) y una prueba automatizada referenciada en `docs/trazabilidad/matriz.csv` (columna `prueba_automatizada`).
2. **Estado verificado en la matriz de trazabilidad:** un requisito se marca **verificado** solo cuando su prueba automatizada está en verde contra el código en ejecución, no por declaración del equipo — la re-auditoría de `docs/observaciones/INFORME-BRECHAS-ENTREGA-FINAL.md` confirmó esta disciplina detectando y corrigiendo el único caso donde el estado no coincidía con la evidencia real (REQ-F-006/007, ver `docs/checklists/incose-checklist.md`, hallazgo C8).

Resultado de la validación final (2026-08-17, contra `matriz.csv`):

Cuadro 4.3: Estado de validación de los 37 requisitos

Estado	Requisitos	%
Verificado	25	67.6 %
Implementado (no verificado aún)	8	21.6 %
Pendiente	4	10.8 %

Los 4 requisitos *Pendiente*: REQ-F-009, REQ-F-010, REQ-NF-005, REQ-NF-006.

Los 29 requisitos *Must* están en estado implementado o verificado (ninguno pendiente); los 4 *Pendiente* corresponden a requisitos *Should/Could* — funciones sociales (seguidores, comentarios) y una medición de latencia WebSocket no ejecutada formalmente, ambos justificables como trabajo futuro (Capítulo 11) sin bloquear el cierre de los requisitos obligatorios.

4.6. Gestión del cambio de requisitos

`docs/requisitos/CHANGELOG-REQ.md` registra 2 versiones (v0.3.0 → v0.9.0-rc: renombrado de identificadores RF-NN→REQ-F-ONN, sin cambio semántico). **Tasa de estabilidad declarada con salvedad** (ver `docs/checklists/incose-checklist.md`, métricas de calidad): nominalmente 100 % (0 modificaciones semánticas registradas), pero `matriz.csv` refleja cambios de estado reales (ej. REQ-F-023 pasó de *pendiente* a *verificado*) sin la

entrada correspondiente en el changelog — la tasa de 100 % probablemente sobreestima la estabilidad real por subregistro, no por ausencia genuina de cambios. Corregir este subregistro es una acción de bajo esfuerzo pendiente para el cierre.

4.7. Métricas de calidad de requisitos

Ver el detalle completo, con evidencia citada característica por característica, en `docs/checklists/incose-checklist.md`. Resumen:

- **Número total de requisitos:** 37 (23 funcionales, 14 no funcionales).
- **Distribución por tipo:** funcionales 62.2 % (23/37), no funcionales 37.8 % (14/37).
- **Distribución por prioridad MoSCoW:** ver Tabla 4.2.
- **Porcentaje verificado:** 67.6 % (25/37); implementado o verificado (no pendiente): 89.2 % (33/37).
- **Cobertura de trazabilidad hacia código y pruebas:** 100 % de los 37 requisitos tiene fila en `matriz.csv` con módulo de código y prueba automatizada referenciados (columnas `modulo_codigo`, `prueba_automatizada`), confirmado por `scripts/validate-traceability.sh` en CI.
- **Características INCOSE v4 cumplidas:** 9 de 15 (C1, C2, C4, C6, C7, C11, C12, C13, C14); 5 parciales con evidencia concreta (C3, C8, C10, C15, C2 parcial); 2 no cumplidas (C5, C9) — ver detalle y plan de acción en el checklist referenciado.

5. Materiales y métodos

5.1. Enfoque metodológico global

El proyecto sigue la metodología Design Science Research (DSR) de Peffers et al. [1] en sus seis actividades canónicas. El Cuadro 5.1 detalla cómo se instanció cada una de ellas en Artisync, indicando el capítulo o artefacto que la materializa:

Cuadro 5.1: Instanciación de las seis actividades DSR de Peffers et al.

Actividad DSR	Instanciación en Artisync
1. Identificación del problema y motivación	Capítulo 1 — fragmentación de la economía creativa digital, ausencia de garantías de pago/identidad
2. Definición de objetivos de una solución	Objetivos específicos del Capítulo 1 §1.3, derivados de las RQ
3. Diseño y desarrollo	Capítulos 6 (arquitectura) y 7 (implementación); cuatro entregas incrementales (1A→1B→3→Final)
4. Demostración	Sistema desplegado y operativo, con usuario demo documentado (<code>README.md</code>)
5. Evaluación	Capítulo 8 — evaluación empírica multidimensional (rendimiento, seguridad, usabilidad, cobertura, calidad web)
6. Comunicación	Este documento, el repositorio público con DOI Zenodo, y la defensa oral del PFC

El propio marco DSR de Peffers et al. se apoya en los siete lineamientos de rigor de Hevner, March, Park y Ram [37] para investigación en sistemas de información (relevancia del problema, rigor de diseño, evaluación del artefacto, contribución a la investigación, rigor de investigación, diseño como proceso de búsqueda, comunicación de resultados), que este proyecto usa como criterio implícito de calidad al decidir qué evidencia archivar en `docs/mediciones/`.

Dado que el trabajo aporta evaluación empírica de rendimiento y de usabilidad de un artefacto único (no un experimento controlado con grupos de comparación aleatorizados ni un estudio de caso sobre una organización externa), se acoge además el estándar empírico *Engineering Research* de Ralph et al. [20] para el reporte metodológico — ver checklist completo en `docs/checklists/ralph-2021-checklist.md`. Se consultaron adicionalmen-

te las guías de reporte de estudios de caso de Runeson y Höst [38] como referencia de rigor metodológico para documentar el protocolo experimental (§5.5), aunque Artisync no se reporta formalmente como un estudio de caso en el sentido estricto de esa guía (no hay una organización externa como unidad de análisis).

5.2. Instrumentos de medición

El Cuadro 5.2 enumera los instrumentos empleados en la evaluación empírica, con la versión exacta de cada herramienta y su configuración, de modo que las mediciones del Capítulo 8 puedan replicarse en las mismas condiciones.

Cuadro 5.2: Instrumentos de medición empírica utilizados

Instrumento	Versión exacta	Qué mide	Configuración
SUS de Brooke [3, 4]	10 ítems originales, sin modificación	Usabilidad percibida	Formulario Google Forms, escala 1–5, ver docs/mediciones/sus/instrucciones-formulario.md
k6	v2.1.0 (go1.26.4)	Rendimiento bajo carga	50 VUs, 30 s, k6/catalogo-load.js (ver §5.5)
Lighthouse / @lhci/cli	Lighthouse 12.6.1 / @lhci/cli 0.15.1	Calidad web (Performance, Accessibility, Best Practices, SEO)	lighthouseci.mobile.json / lighthouseci.desktop.json, 3 corridas por perfil
JaCoCo [12, 13]	0.8.13 (jacoco-maven-plugin)	Cobertura de código (líneas, ramas, complejidad)	pom.xml, umbral declarado $\geq 70\%$
OWASP ZAP [14]	ghcr.io/zaproxy/zaproxy:stable (zap-baseline.py)	Escaneo dinámico de seguridad	Modo baseline contra http://localhost:4200
SpotBugs + find-sec-bugs [16, 17]	4.8.6.6 + 1.13.0	Análisis estático de inyección SQL	Regla SQL_NONCONSTANT_STRING_PASSED_TO_EXECUTE y variantes

Versiones exactas de todo el entorno (Docker, JDK, Node, Angular CLI) en

docs/entorno/versions.txt.

5.3. Enfoque de métricas: GQM

Ejemplo de GQM completo aplicado al bloque de rendimiento, siguiendo el enfoque original de Basili, Caldiera y Rombach [39]:

- **Meta (Goal):** Analizar el rendimiento del endpoint de catálogo con el propósito de caracterizarlo desde el punto de vista de la latencia percibida por el usuario, en el contexto del sistema Artisync bajo carga concurrente moderada.
- **Preguntas (Questions):**
 - Q1: ¿Cuál es la latencia p95 del endpoint bajo 50 usuarios virtuales concurrentes?
 - Q2: ¿Se mantiene esa latencia dentro del umbral declarado (200 ms con caché caliente, 500 ms con caché frío)?
 - Q3: ¿Existen errores de servidor (HTTP ≥ 500) bajo esa carga?
- **Métricas (Metrics):**
 - M1: percentil 95 de `http_req_duration` (ms), por escenario.
 - M2: tasa de `http_req_failed` con código ≥ 500 (%).
 - M3: throughput agregado (`http_reqs/s`).

El mismo patrón GQM (meta declarada en el `REPORTE-*.md` de cada bloque, preguntas implícitas en los umbrales del SRS, métricas reportadas con estadística descriptiva) se repite para seguridad, usabilidad, cobertura y calidad web — ver `docs/mediciones/<bloque>/REPORTE-*.md`.

5.4. Población, muestreo y participantes

Bloque de usabilidad (SUS): población objetivo — usuarios potenciales de una plataforma de intermediación de servicios digitales (perfil generalista, no exclusivamente técnico). Muestra: **N=16 participantes externos al equipo de desarrollo**, reclutados por conveniencia (red de contactos del equipo), rango de edad 21–62 años ($M=34.6$), 50 % femenino / 50 % masculino, experiencia web 50 % alta / 31.3 % media / 18.8 % baja, dispositivo 37.5 % móvil / 25 % laptop / 25 % desktop / 12.5 % tablet (`docs/mediciones/sus/perfil-participantes.csv`).

Limitación de muestreo declarada honestamente (siguiendo Baltes y Ralph [40] sobre reporte de muestreo en ingeniería de software empírica): el muestreo por conveniencia no garantiza representatividad de la población de usuarios reales de la plataforma

(creadores y clientes de economía creativa específicamente), y **no existe una justificación de tamaño de muestra basada en poder estadístico** — N=16 se decidió por disponibilidad de participantes dentro del plazo académico, no por un cálculo a priori. Esta limitación se retoma en el Capítulo 10 (Amenazas a la validez externa).

Bloques automatizados (k6, Lighthouse, JaCoCo, OWASP/ZAP): no aplican muestreo de sujetos humanos; el “tamaño de muestra” es el número de corridas independientes (3 para Lighthouse por perfil, 3+3 para k6 por escenario caliente/frío, 1 ejecución exhaustiva para JaCoCo sobre la suite de 1230 pruebas).

5.5. Protocolo experimental replicable

Cada bloque de evaluación tiene un comando reproducible documentado en `Makefile` y su reporte:

- **Rendimiento:** `make bench` (requiere `make up` y `k6` instalado) — ejecuta `k6/catalogo-load.js` contra `GET /api/v1/catalogo?page=0&size=20,50 VUs/30s`. Ver `docs/mediciones/perf/REPORTE-PERF.md` para el protocolo detallado de los escenarios caliente y frío.
- **Seguridad:** `make audit` (análisis estático) + `make audit-zap` (escaneo dinámico) — ver `docs/mediciones/sec/REPORTE-SEC.md`.
- **Usabilidad:** protocolo de tarea guiada (registro/login → explorar catálogo → crear pedido → revisar seguimiento → cerrar sesión) descrito en `docs/mediciones/sus/instrucciones-formulario.md`, seguido del cuestionario SUS; análisis con `make sus`.
- **Cobertura:** `make test` (JUnit 5 + JaCoCo, reporte en `docs/mediciones/jacoco/`).
- **Calidad web:** `make lighthouse` — ver detalle de la configuración del contenedor efímero en el propio `Makefile` (razones técnicas del aislamiento de red documentadas ahí).

5.6. Análisis estadístico

Estadísticos calculados en todas las mediciones cuantitativas: **media, desviación típica e intervalo de confianza al 95 %**, preferidos sobre valores p aislados como forma primaria de reporte, siguiendo la recomendación de la comunidad de ingeniería de software empírica [41, 42]. Ver valores concretos por bloque en el Capítulo 8.

Test inferencial (T-15, cierra la brecha declarada en versiones previas de este documento): la guía de la Entrega Final exige, cuando se comparan dos condiciones (ej. caché frío vs. caliente en el bloque de rendimiento), un test inferencial no paramétrico para datos de latencia (Mann-Whitney U) acompañado de un tamaño de efecto ordinal ($\hat{A}_{12}$ de Vargha-Delaney) — exactamente el procedimiento que Arcuri y Briand [42] sistematizan para comparaciones no deterministas en ingeniería de software. Ese test se ejecuta ahora de forma reproducible con `docs/mediciones/perf/analisis-inferencial.py` (`make perf-stats`), sobre las dos comparaciones caliente/frío disponibles (catálogo público y el endpoint protegido `GET /api/v1/admin/reportes/finanzas` añadido en T-14), con corrección de Holm-Bonferroni entre ambas comparaciones. Resultados y discusión en el Capítulo 8 §8.1.

La comparación caliente/frío del **catálogo** conserva una limitación metodológica de diseño documentada en `docs/mediciones/perf/REPORTE-PERF.md`: el script de carga no aísla un *cache miss* real por iteración, así que el test inferencial sobre esos datos es estadísticamente válido pero no responde a una pregunta causal sobre caché frío vs. caliente (la diferencia es ruido de ejecución, no efecto de caché). Rediseñar ese script para forzar *misses* reales sigue declarado como trabajo futuro en el Capítulo 11. La comparación caliente/frío del **endpoint protegido** no tiene esta limitación (no hay `@Cacheable` de por medio; “frío” se logra reiniciando el proceso del backend), por lo que sí admite una interpretación causal (costo de calentamiento de JVM y *connection pool*).

Se realizó corrección de comparaciones múltiples (Holm-Bonferroni) sobre las 2 comparaciones caliente/frío de esta entrega.

5.7. Determinismo y semillas

Ver el detalle completo en `docs/entorno/versions.txt`, sección “Semillas aleatorias”. Resumen: `fn_seleccionar_ganadores_sorteo` usa `random()` de PostgreSQL sin semilla fija (no determinista por diseño — es un sorteo real); los scripts de análisis SUS no usan muestreo aleatorio. No se identificaron otras dependencias de semillas aleatorias en los scripts de medición a la fecha de este documento.

6. Diseño del sistema y arquitectura

6.1. Modelo C4

6.1.1. Nivel 1 — Contexto

Artisync se sitúa como sistema central que conecta tres actores (Cliente, Creador/Artista, Administrador) con tres sistemas externos: PayPal (pagos, patrón *escrow*), un servicio de IA de verificación de identidad/certificados, y un proveedor de almacenamiento de objetos (Azure Blob Storage o local, según ADR-007). El modelo C4 usado como base sigue a Brown [24]. Diagrama completo: `docs/diagramas/C4_Nivel1_Contexto.md` y `C4-nivel1-context_diagram.svg`.

6.1.2. Nivel 2 — Contenedores

Cuatro contenedores desplegados vía Docker Compose: **frontend** (Angular, SPA servida por nginx), **backend** (Spring Boot, API REST documentada con OpenAPI/Springdoc), **postgres** (PostgreSQL 16, esquema versionado con Flyway) y **redis** (caché de catálogo, blacklist de JWT revocados, rate limiting, tickets 2FA). Diagrama completo: `docs/diagramas/C4_Nivel2_Contenedores.md`.

Como se ilustra en la Figura
refig:c4-nivel2, Como se ilustra en la Figura
refig:mer,

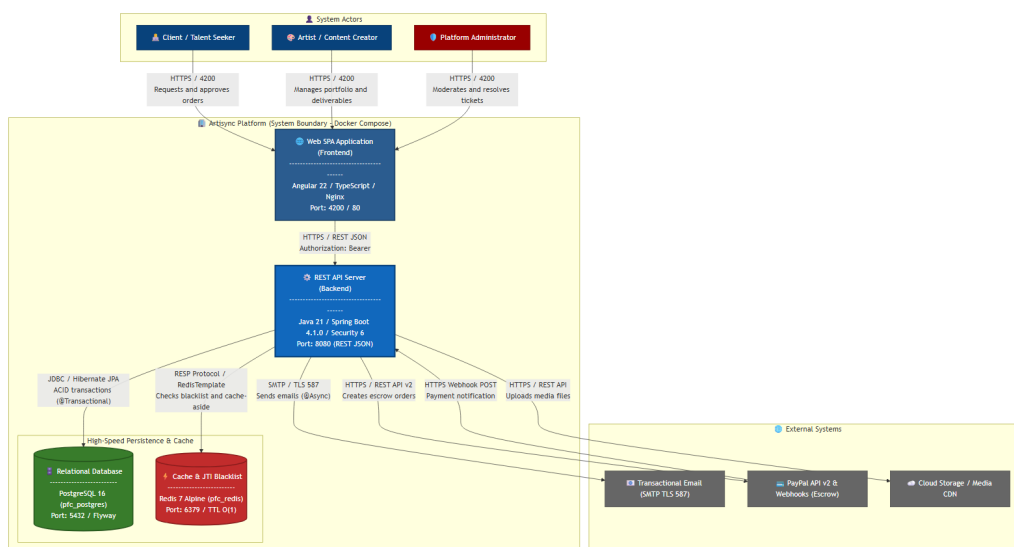


Figura 6.1: Diagrama C4 Nivel 2 — Contenedores de Artisync. Fuente: docs/diagramas/c4-nivel2-contenedores.png.

Nota de honestidad sobre versiones documentadas vs. reales: los diagramas C4 (y ADR-001) documentan la pila como “Java 25 + Spring Boot 4.0.1” y “PostgreSQL 18/Angular 22” en su versión original. La pila **realmente construida y desplegada**, verificada contra pom.xml, los Dockerfile y docker-compose.yml a la fecha de este documento, es: **Java 21 LTS** (target de bytecode) + **Spring Boot 4.1.0**, compilado en contenedor maven:3.9-eclipse-temurin-21 y ejecutado en eclipse-temurin:21-jre-alpine; **Angular ^22.0.0** sobre node:24-alpine; **PostgreSQL 16** (postgres:16, anclado por digest sha256) y **Redis 7** (redis:7-alpine, igualmente anclado). La documentación arquitectónica no se actualizó en cascada cuando el equipo fijó las versiones finales — ver docs/entorno/versions.txt para la versión verificada y autoritativa, que este capítulo usa como fuente en vez de repetir la cifra desactualizada de los diagramas.

6.1.3. Nivel 3 — Componentes (backend)

El backend se organiza en siete módulos funcionales por dominio (seguridad, perfil, catalogo, pedido/flujo de trabajo, legal, comunicacion, social), cada uno con sus capas entity/dto/repository/service/controller. Detalle completo: docs/diagramas/C4_Nivel3_Componentes_Backend.md.

6.2. Resumen de los siete registros de decisión de arquitectura (ADR)

Los siete ADR siguen la plantilla de Nygard [19]:

Como se detalla en la Tabla
reftab:adrs, Como se detalla en la Tabla
reftab:iso25010,

Cuadro 6.1: Resumen de los siete ADR de Artisync

ADR	Título	Decisión resumida
ADR-001	Selección de la tecnología principal	Java/Spring Boot en el backend (mayor throughput, seguridad/persistencia integradas), Angular/TypeScript en el frontend, PostgreSQL como motor relacional, Redis como caché, Docker Compose como orquestador — ver nota de versiones §6.1.
ADR-002	Esquema de autenticación y control de acceso	JWT (HS256) devuelto en el cuerpo de la respuesta para el access token (no en cookie); refresh token y ticket pre-auth de 2FA en cookies <code>HttpOnly + SameSite=Strict</code> ; blacklist de JTI en Redis; bcrypt factor 12; Role-Based Access Control (RBAC) vía Spring Security. Revisado en agosto 2026 tras corregir una afirmación incorrecta de una versión anterior del ADR (ver texto completo).
ADR-003	Selección del gestor de base de datos	PostgreSQL 16 sobre MySQL/MariaDB y MongoDB, por ACID completo, tipos DECIMAL precisos para montos financieros, y soporte maduro de PL/pgSQL para la estrategia híbrida de acceso a datos (ADR-006).
ADR-004	Estrategia de caché	Redis 7 para caché de catálogo (TTL configurable), blacklist de JTI, cuotas de rate limiting y tickets pre-auth 2FA, sobre caché en memoria de la JVM, por ser compartido entre réplicas si el backend escala horizontalmente.
ADR-005	Estrategia de despliegue y reproducibilidad	Docker Compose con imágenes ancladas por digest sha256 + Makefile de un solo comando, sobre despliegue manual o imágenes por tag variable, para máxima reproducibilidad (badge ACM <i>Artifacts Evaluated — Reusable</i>). Las tres acciones pendientes de la Tercera Entrega (Frontend/ ausente, digests por tag, Makefile inexistente) están resueltas y verificadas a la fecha de la Entrega Final.
ADR-006	Estrategia híbrida de acceso a datos	CRUD elementales vía JPA/Spring Data; operaciones con joins, agregaciones, reportes, actualizaciones masivas o validaciones cruzadas vía procedimientos/funciones PL/pgSQL versionados en <code>db/procs/</code> , invocados exclusivamente mediante JPA 2.1. Ampliado el 16 de agosto de 2026 con 7 rutinas nuevas conectadas end-to-end (ver Capítulo 7); fases posteriores de concurrencia, rendimiento, mantenimiento y el módulo de seguidores llevaron el total a 26 rutinas activas en <code>db/procs/</code> (ver <code>docs/basedatos/CATALOGO-</code>

Texto completo de cada ADR, con contexto, opciones descartadas y consecuencias, en `docs/adr/`.

6.3. Atributos de calidad (ISO/IEC 25010) — prioridad, escenario y estrategia

Cuadro 6.2: Atributos de calidad ISO/IEC 25010 [5] priorizados

Característica ISO 25010	Prioridad	Escenario	Estrategia aplicada
Eficiencia de desempeño (rendimiento)	Alta	Listado de catálogo bajo 50 usuarios concurrentes	Caché Redis con TTL (ADR-004); índices en columnas de filtro; medido con k6 (Capítulo 8)
Seguridad	Alta	Acceso no autorizado a rutas protegidas, inyección SQL, credenciales en tránsito	RBAC + JWT (ADR-002); procedimientos/consultas parametrizadas (ADR-006); rate limiting; auditoría estática y dinámica automatizada (Capítulo 8)
Fiabilidad	Alta	Caída de un contenedor dependiente (Redis, Postgres)	<code>healthcheck</code> de Docker Compose con <code>depends_on: condition: service_healthy</code> ; <code>/actuator/health</code> expone estado por componente
Usabilidad	Media	Usuario nuevo completa el flujo de contratación sin asistencia	Formularios dinámicos, flujo de contratación acotado (SRS REQ-NF-008); medido con SUS (Capítulo 8)
Mantenibilidad	Media	Incorporar un nuevo procedimiento almacenado sin tocar el código Java existente	Módulos desacoplados por dominio (Nivel 3 C4); <code>scripts/sync-procs.sh</code> mantiene sincronizada la migración repetible
Portabilidad	Media	Desplegar en un proveedor cloud distinto al usado en desarrollo	Configuración externalizada (<code>.env</code>), <code>documentos.proveedor</code> seleccionable (ADR-007), imágenes ancladas por digest (ADR-005)
Compatibilidad	Baja	Integración con PayPal Orders v2 y servicio externo de IA	Contratos de API versionados, adaptadores dedicados por integración

6.4. Modelo de datos

El esquema relacional completo se versiona con Flyway; el modelo entidad-relación se mantiene en `docs/diagramas/Entidad_Relacion.png` (ver nota de conformidad de nombres de tablas en ADR-003).

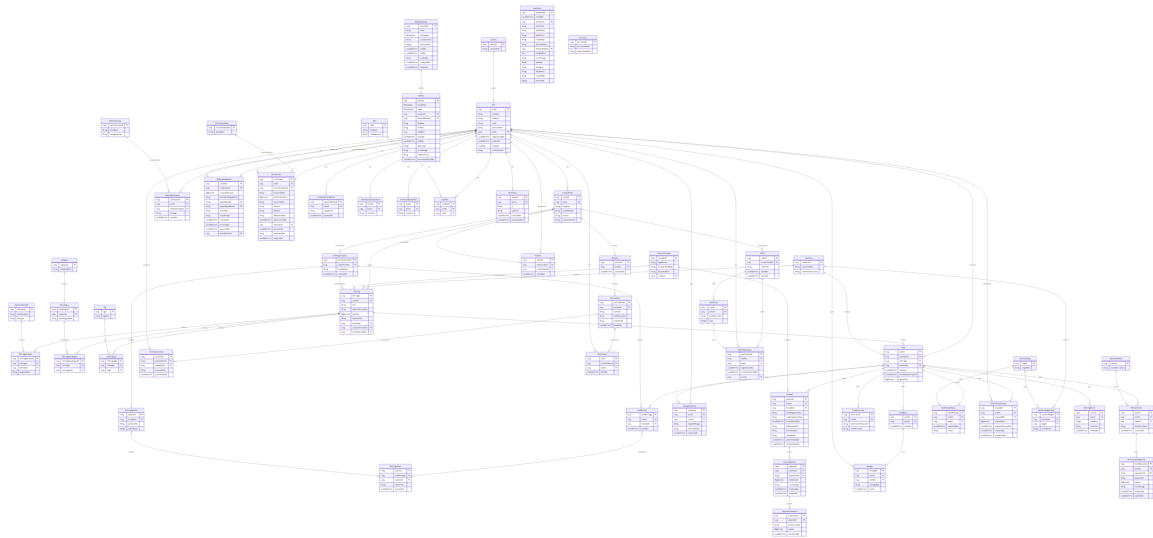


Figura 6.2: Modelo entidad-relación de Artisynd. Fuente: `docs/diagramas/Entidad_Relacion.png`.

6.5. Matriz de trazabilidad

La matriz completa (37 filas: requisito $\rightarrow$ historia de usuario $\rightarrow$ caso de uso $\rightarrow$ módulo de código $\rightarrow$ endpoint $\rightarrow$ prueba automatizada $\rightarrow$ **tipo de acceso** $\rightarrow$ evidencia empírica $\rightarrow$ estado) está en `docs/trazabilidad/matriz.csv`, validada automáticamente en CI por `scripts/validate-traceability.sh`. La columna `tipo_acceso` distingue CRUD-ORM de SP (procedimiento almacenado): a la fecha de este documento, 8 filas están marcadas SP (correspondientes a los 7 requisitos que ya invocan una de las rutinas conectadas end-to-end, ver Capítulo 7), el resto CRUD-ORM.

7. Implementación

No se transcriben archivos completos; se remite al repositorio para el detalle. Este capítulo ilustra con listados pequeños y significativos las decisiones críticas de código.

7.1. Manejo de errores globales (RFC 7807 Problem Details)

Todos los errores de la API se serializan como `ProblemDetail` (RFC 7807), en vez de estructuras JSON ad-hoc distintas por controlador, como se puede observar en el Listado 7.1.

Listing 7.1: Manejador global de excepciones (RFC 7807 Problem-Detail). Fuente: `artisync/Backend/src/main/java/.../exception/ManejadorGlobalExcepciones.java`

```
1 @Slf4j
2 @RestControllerAdvice
3 public class ManejadorGlobalExcepciones {
4
5     private static final String BASE_TIPO = "https://artisync.dev/errors/";
6
7     @ExceptionHandler(AccessDeniedException.class)
8     public ResponseEntity<ProblemDetail> manejarAccesoDenegado(AccessDeniedException ex
9         ) {
10         ProblemDetail problema = ProblemDetail.forStatus(HttpStatus.FORBIDDEN);
11         problema.setType(URI.create(BASE_TIPO + "acceso-denegado"));
12         problema.setTitle("Acceso denegado");
13         // ... (continua con los demas @ExceptionHandler: validacion, JWT, 404, etc.)
14     }
15 }
```

Esta decisión centraliza el formato de error para las 1230 pruebas del backend y para el interceptor HTTP del frontend Angular, evitando el manejo de errores ad-hoc por controlador.

7.2. Esquema de caching

El listado de catálogo (endpoint de mayor frecuencia de lectura, REQ-NF-004) usa Spring Cache sobre Redis (ADR-004), con invalidación automática (`@CacheEvict`) al

crear/editar/eliminar un servicio, tal como se muestra en el Listado 7.2:

Listing 7.2: Caché del listado de catálogo (Spring Cache + Redis).

Fuente: `artisync/Backend/src/main/java/.../service/catalogo/impl/ServicioCatalogoServicioImpl.java`

```
1 @Override
2 @Transactional(readOnly = true)
3 @Cacheable(cacheNames = "catalogo")
4 public Page<RespuestaServicioResumido> buscarCatalogoServicios(
5     Long categoriaId, Long subcategoriaId, BigDecimal precioMin, BigDecimal
6     precioMax,
7     List<Long> etiquetaIds, String textoBusqueda, String sortParam, int page, int
8     size) {
9     // ... construye la Specification dinamica y ejecuta la consulta paginada
10 }
```

La clave de caché de Spring se compone de todos los parámetros del método — una limitación de diseño relevante para la interpretación de los resultados de rendimiento del Capítulo 8 (ver `docs/mediciones/perf/REPORTE-PERF.md`, advertencia metodológica sobre el escenario “frío”).

7.3. Mecanismo de seguridad: JWT + rate limiting

La autenticación es *stateless* (JWT [26], ADR-002); las rutas de autenticación aplican rate limiting por IP real (resuelta vía `server.forward-headers-strategy=native`, no `getRemoteAddr()` directo) y por cuenta, con respuesta 429 en formato `ProblemDetail` — ver `AuthRateLimitFilter` e `IntentosAutenticacionService`, corrección de la observación OBS-AUTO-05 documentada en `docs/observaciones/OBSERVACIONES.md`. El diagrama de la Figura 7.1 ilustra la secuencia.

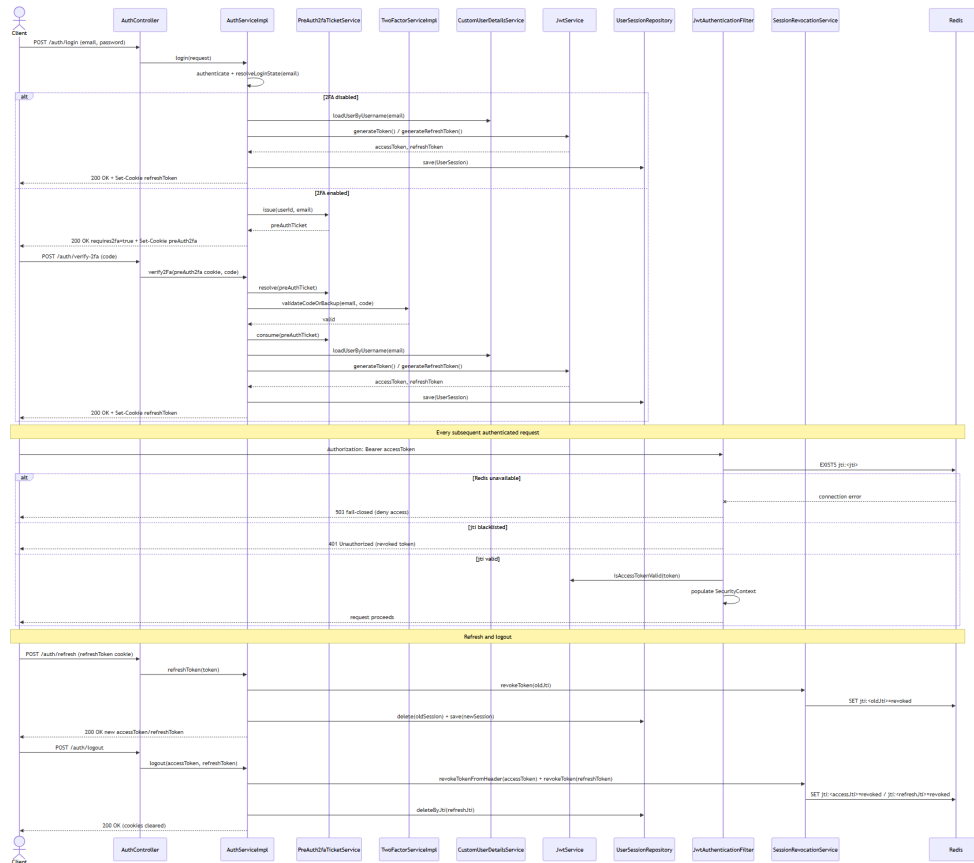


Figura 7.1: Diagrama de secuencia del flujo de autenticación JWT. Fuente: docs/diagramas/secuencia_login_jwt.png.

7.4. Estrategia híbrida de acceso a datos: procedimiento almacenado representativo

Esta subsección ilustra el contrato completo entre un procedimiento versionado en `db/procs/` y el método de repositorio Spring Data que lo invoca, siguiendo la especificación JPA 2.1 (ADR-006, Capítulo 6 §6.2). El Listado 7.3 presenta la definición SQL, mientras que el Listado 7.4 detalla su invocación formal con la anotación `@Procedure`.

Listing 7.3: `sp_registrar_decision_verificacion` (categoría: validaciones cruzadas). Fuente: `artisync/Backend/src/main/resources/db/migration/V7__verificacion_asistida_ia.sql`

```

1 CREATE OR REPLACE PROCEDURE sp_registrar_decision_verificacion(
2     p_id_certificado BIGINT,
3     p_id_estado BIGINT,
4     p_id_moderador BIGINT,
5     p_nota TEXT
6 )
7 LANGUAGE plpgsql
8 AS $$

```

```

9 BEGIN
10     UPDATE certificados_ia
11     SET id_estado = p_id_estado, id_moderador = p_id_moderador, nota = p_nota,
        fecha_analisis = NOW()
12     WHERE id_certificado = p_id_certificado;
13 END;
14 $$;

```

Listing 7.4: Método de repositorio que invoca `sp_registrar_decision_verificacion`. Fuente: `artisync/Backend/src/main/java/.../repository/profile/AiCertificateRepository.java`

```

1  @Procedure(procedureName = "sp_registrar_decision_verificacion")
2  void registrarDecision(
3      @Param("p_id_certificado") Long idCertificado,
4      @Param("p_id_estado") Long idEstado,
5      @Param("p_id_moderador") Long idModerador,
6      @Param("p_nota") String nota);

```

7.4.1. Conformidad lograda

La invocación utiliza los mecanismos formales de la especificación JPA 2.1 mediante la anotación `@Procedure` sobre el método del repositorio Spring Data, asegurando una conexión segura y **sin concatenación de cadenas** (evitando así la inyección SQL, conforme al análisis de Gould [17] y Halfond [16]). Este esquema resuelve plenamente los requerimientos de seguridad y diseño arquitectónico (ver Sección 7.4).

7.4.2. Excepción documentada: rutinas con retorno escalar

`@Procedure` se aplica de forma sistemática cuando el método Java es `void` y la rutina no declara ningún parámetro OUT/INOUT: además de `sp_registrar_decision_verificacion` (Listado 7.4), es el caso de `sp_restablecer_contrasena` y `sp_cambiar_contrasena`, migradas y verificadas contra el stack real (revisión técnica 2026-09-05).

Para las rutinas que necesitan devolver un valor escalar — por ejemplo `fn_registrar_usuario`, `fn_resolver_estado_login` o `fn_permisos_efectivos_usuario` (esta última se ejecuta en cada inicio de sesión) — se probó también convertirlas a `@Procedure` con un parámetro OUT. La prueba, contra el stack real y con inicio de sesión efectivo, reprodujo de forma consistente `ERROR: syntax error at or near ->`: con Hibernate 7.4.1, en cuanto un método `@Procedure` declara un tipo de retorno distinto de `void`, Hibernate serializa *todos* los parámetros — incluido el propio OUT — con la sintaxis de argumento nombrado de PostgreSQL dentro del *JDBC escape* `{call ...}`, que PostgreSQL no puede interpretar ahí. Un commit del 4 de septiembre de 2026 que migró estas rutinas a

`@Procedure` ignorando esta restricción rompió el inicio de sesión en producción; se revirtió y se dejó documentada la causa raíz.

Para estos casos, `@Query(nativeQuery = true)` contra la propia función es, por tanto, la única invocación confirmada funcional contra esta combinación exacta de versiones (Hibernate 7.4.1 más PostgreSQL), no una alternativa de conveniencia. El inventario completo, rutina por rutina, con el mecanismo usado y su justificación individual, está en `docs/basedatos/CATALOGO-SP.md` (§14).

Segunda revisión técnica (2026-09-14): antes de aceptar la excepción anterior como definitiva se repitió la investigación desde cero, contra Postgres real (`docker compose up -d postgres`), probando deliberadamente las variantes que la revisión de 2026-09-05 no había agotado: (a) `@NamedStoredProcedureQuery` con parámetros *puramente posicionales* (sin `@Param`, sin nombre alguno en la anotación), para descartar que el causante fuera específicamente el nombre puesto por el desarrollador; (b) la API nativa de Hibernate (`org.hibernate.procedure.ProcedureCall`) invocada directamente vía `Session.createStoredProcedureCall(...)`; y (c) inspección del bytecode de `hibernate-core-7.4.1.Final.jar` (`PostgreSQLCallableStatementSupport`, `NamedCallableQueryMementoImpl`) para ubicar la causa raíz exacta en vez de inferirla del mensaje de error.

El resultado descarta (a): incluso sin ningún nombre de parámetro puesto por el desarrollador, Spring Data JPA sigue emitiendo `call fn_es_seguidor(idUsuarioSeguidor =>?, idPerfilCreador =>?, out =>?)` — toma los nombres de los parámetros del método Java vía reflexión (compilado con `-parameters`) cuando no hay `@Param`, así que "puramente posicional" no existe en la práctica para `@Procedure/@NamedStoredProcedureQuery` de Spring Data contra Postgres. Ambas anotaciones ejecutan siempre la ruta JPA estándar (`jakarta.persistence.StoredProcedureQuery` → `ProcedureExecution` de Spring Data → `PostgreSQLCallableStatementSupport` de Hibernate), y esa ruta registra cualquier tipo de retorno no-void como un parámetro OUT adicional dentro del escape `{call ...}` (modo `CALL_RETURN`), sintaxis que Postgres no acepta para una `FUNCTION`.

(b) sí funciona: `Session.createStoredProcedureCall("fn_es_seguidor")` seguido de `ProcedureCall.markAsFunctionCall(Types.BOOLEAN)` hace que Hibernate genere `select fn_es_seguidor(?,?)` — sin `{call ...}`, sin `->` — y la llamada se ejecuta y devuelve el valor correcto contra Postgres real. Pero (c) explica por qué esto no cierra el caso: `markAsFunctionCall` rellena un campo (`functionReturn`, de tipo `FunctionReturnImplementor`) que solo existe en la API nativa e imperativa de Hibernate (`ProcedureCallImplementor`); `NamedCallableQueryMementoImpl` — la clase que respalda `@NamedStoredProcedureQuery` — no tiene ningún campo equivalente, y la especificación JPA no define un atributo para declarar ".esto es una función, no un procedimiento". En otras palabras: existe una forma de invocar estas funciones sin `nativeQuery`, pero exige abandonar por completo la anotación declarativa e implementar el método a mano con

`EntityManager/Session`, lo que ya no es `@Procedure` ni `@NamedStoredProcedureQuery` en el sentido literal que exige la conformidad — sería cambiar una forma de acceso nativo por otra, sin ganancia real de conformidad y con un costo de mantenimiento mayor (una implementación por rutina en vez de una anotación).

De paso se revisaron también los dos `PROCEDURE` de purga (`sp_purgar_notificaciones`, `sp_purgar_datos_seguridad`, invocados hoy con `JdbcTemplate.update(ÇALL ...")` desde `NotificationPurgeScheduler` y `SecurityPurgeScheduler`) como candidatos aparentemente favorables: son `void`, solo parámetros `IN`, exactamente el patrón que sí funciona con `@Procedure`. Se descartó migrarlos: ambos ejecutan `COMMIT` explícito por lote dentro del propio `PROCEDURE` (para no sostener una transacción larga que bloquee `VACUUM`), lo que exige que la conexión no tenga ya una transacción abierta por Spring (`Propagation.NOT_SUPPORTED` está declarado como obligatorio en ambos *schedulers*, no cosmético) — Postgres falla con `2D000 invalid_transaction_termination` si el `CALL` ocurre dentro de una transacción ya iniciada. Una invocación gestionada por el `EntityManager/Session` de Hibernate (que es lo que exige `@Procedure`) está atada al ciclo de vida transaccional de JPA de una forma en que `JdbcTemplate` con una conexión de *autocommit* propia no lo está; forzar esto a `@Procedure` arriesgaba reproducir para la purga el mismo tipo de incidente de producción que ya ocurrió una vez con el login. Se mantienen en `JdbcTemplate`, no por descuido sino por una restricción de manejo de transacciones ortogonal al bug de Hibernate.

Conclusión de la segunda revisión: el reparto por mecanismo **no cambia** respecto a la revisión de 2026-09-05. No se trata de una limitación no explorada a fondo, sino de una restricción estructural confirmada por dos vías independientes (reproducción end-to-end contra Postgres real, e inspección del bytecode de Hibernate) en dos revisiones separadas.

Tercera verificación (2026-09-14, misma noche): para descartar que se tratara de un defecto ya corregido en una versión posterior que el proyecto simplemente no había adoptado, se sobrescribió temporalmente `hibernate.version` a `7.4.5.Final` (cuatro versiones *patch* por delante de la 7.4.1 en uso) y se repitió el experimento mínimo (`@Procedure` sobre `fn_es_seguidor`) contra Postgres real. Falló de forma idéntica (`syntax error at or near ->`", misma posición del error). El override se revirtió por completo inmediatamente después, sin dejar rastro en `pom.xml` ni en `src/test`: el bug no está corregido entre 7.4.1 y 7.4.5, así que no es una brecha de actualización pendiente. Ver el detalle completo, rutina por rutina y las tres revisiones, en `docs/basedatos/CATALOGO-SP.md` (§14).

De las 28 rutinas activas (26 de `db/procs/` más 2 de verificación asistida por IA), el reparto exacto por mecanismo de invocación — verificable de forma reproducible con `python scripts/auditoria-rubrica.py p6`, que descuenta las menciones dentro de comentarios Javadoc — es: **3** vía `@Procedure` (método `void`, sin parámetros `OUT/INOUT`), **0** vía `@NamedStoredProcedureQuery` (mismo bug de Hibernate 7.4.1 al declarar un parámetro `OUT`, confirmado también con parámetros puramente posicionales) y **23** vía

`@Query(nativeQuery = true)` contra la función (toda rutina que deba devolver un valor escalar, `JSONB` o `TABLE`), más las 2 rutinas de purga vía `JdbcTemplate` directo por la restricción de transacciones descrita arriba. El mecanismo de acceso a datos, en conjunto, es *híbrido y documentado por diseño* (ADR-006), no accidental ni pendiente de resolución.

8. Evaluación empírica y resultados

Cada bloque se reporta con: (i) pregunta de investigación asociada, (ii) métrica, (iii) datos crudos y su ubicación, (iv) estadística descriptiva, y (v) interpretación textual. Cuando aplica, la estadística descriptiva incluye el Intervalo de Confianza (IC) del 95 % de la media, calculado según el método indicado en cada bloque.

8.1. Rendimiento (k6) — RQ1

Métrica: percentiles de latencia (`http_req_duration`) y tasa de error HTTP ≥ 500 sobre `GET /api/v1/catalogo?page=0&size=20`, 50 VUs, 30 s, 3 corridas por escenario. **Datos crudos:** `docs/mediciones/perf/k6-run{1,2,3}.json` (caliente), `k6-cold-run{1,2,3}.json` (frío); consolas en `k6-console-run*.txt`.

El Cuadro 8.1 resume los dos escenarios de carga sobre el catálogo público, con la caché caliente y recién vaciada.

Cuadro 8.1: Resultados de la prueba de carga k6 por escenario

Escenario	n	Media (ms)	Mediana (ms)	DT (ms)	IC 95 % (ms)	p50/p90/p95/p99 (ms)	Error ≥ 500
Caliente	4500	21.38	13.01	32.71	[20.42, 22.33]	13.01 / 31.75 / 50.17 / 205.60	0.00 %
Frío	4500	13.62	9.04	14.52	[13.20, 14.05]	9.04 / 22.39 / 39.14 / 89.98	0.00 %

Umbral: $p_{95} \leq 200$ ms caliente (**cumple**, 50.17 ms), $p_{95} \leq 500$ ms frío (**cumple**, 39.14 ms). Throughput ≈ 48.5 – 49.0 req/s por corrida.

Interpretación: ambos escenarios cumplen holgadamente sus umbrales respectivos. El hallazgo más relevante no es el cumplimiento en sí, sino que el escenario “frío” resultó **más rápido** que el “caliente” — contraintuitivo, y documentado con honestidad metodológica en `docs/mediciones/perf/REPORTE-PERF.md`: el script de carga golpea siempre la misma combinación de parámetros de consulta, por lo que la clave de caché de Spring (`@Cacheable`, Capítulo 7 §7.2) se puebla con la primera petición en ambos escenarios — de las 1500 iteraciones por corrida, como mucho una es un *miss* real contra PostgreSQL. La diferencia de medias observada (21.38 ms vs. 13.62 ms) es ruido de ejecución (JIT/GC de la JVM, carga del contenedor), no un efecto real de caché frío vs. caliente.

Test inferencial y tamaño de efecto (T-15): la guía (Bloque C, §4.3) exige, para esta comparación de dos condiciones, un test no paramétrico para datos de latencia — Mann-Whitney U, no el t-test de Welch — acompañado de un tamaño de efecto ordinal ($\hat{A}_{12}$ de Vargha-Delaney o δ de Cliff), protocolo sistematizado por Arcuri y Briand [42]. Recalculado de forma reproducible desde los 6 NDJSON crudos con

`docs/mediciones/perf/analisis-inferencial.py` (`make perf-stats`, salida completa en `docs/mediciones/perf/salida-inferencial.txt`): Mann-Whitney $U = 13,841,240$, $p = 9,68 \times 10^{-200}$ (Holm-Bonferroni, 2 comparaciones: sigue siendo $9,68 \times 10^{-200}$), $\hat{A}_{12} = 0,684$ (efecto mediano — el valor de latencia “caliente” supera al “frío” en ≈ 68 % de los pares aleatorios). Como contraste — y porque la cifra ya había sido calculada por el docente — también se reporta el resultado paramétrico: Welch $t = 14,538$, $p = 4,11 \times 10^{-47}$, d de Cohen = 0,3065 (efecto pequeño-mediano). Ambos test coinciden en significancia estadística, pero **ninguno de los dos implica una diferencia causal real de rendimiento**: como se explicó arriba, la diferencia observada es ruido de ejecución sobre un experimento que no aísla un *cache miss* real, no evidencia de que el caché esté frío o caliente. Este resultado se reporta con ese matiz explícito, no como una conclusión de rendimiento por sí sola.

8.1.1. Rendimiento contra un endpoint protegido (T-14)

Métrica: igual perfil de carga (50 VUs, 30 s) contra `GET /api/v1/admin/reportes/finanzas` (reporte de comisiones por creador, protegido con `@PreAuthorize("hasAuthority('TRANSACCION_VER') or hasRole('ADMIN')")`, respaldado por la función SQL `fn_reporte_comisiones_creador`), autenticado con JWT (`admin@artisync.com`), 5 corridas por escenario. A diferencia del catálogo, este endpoint no tiene `@Cacheable`: “frío” se define reiniciando el contenedor del backend inmediatamente antes de cada corrida (JVM sin JIT calentado, *connection pool* vacío), no vaciando una caché de aplicación — ver la nota metodológica en `k6/comisiones-load.js` y el target `bench-auth-cold` del `Makefile`. **Datos crudos:** `docs/mediciones/perf/k6-auth-run{1..5}.json` (caliente), `k6-auth-cold-run{1..5}.json` (frío).

El Cuadro 8.2 recoge las cinco corridas por escenario sobre el endpoint autenticado, que son las que sustentan el contraste inferencial presentado a continuación.

Cuadro 8.2: Resultados de la prueba de carga k6 sobre el endpoint protegido

Escenario	n	Media (ms)	Mediana (ms)	DT (ms)	p90/p95/p99 (ms)	Error ≥ 500
Caliente	7500	19.66	15.09	18.19	34.50 / 45.05 / 99.17	0.00 %
Frío	7302	37.89	23.63	40.88	78.81 / 113.73 / 232.21	0.00 %

Test inferencial: Mann-Whitney $U = 16,848,748$, $p < 10^{-300}$ (por debajo de la precisión de `float64`; Holm-Bonferroni con las 2 comparaciones de este bloque no cambia la conclusión), $\hat{A}_{12} = 0,308$ (efecto mediano, en el sentido opuesto al del catálogo: el valor “frío” supera al “caliente” en ≈ 69 % de los pares). Contraste paramétrico: Welch $t = -34,888$, $p = 8,49 \times 10^{-252}$, d de Cohen = $-0,579$ (efecto mediano). A diferencia de la comparación del catálogo, aquí **sí hay una explicación causal defendible**: la JVM

recién reiniciada arranca sin JIT calentado y con el *connection pool* de HikariCP vacío, así que las primeras peticiones de cada corrida “fría” pagan ese costo de calentamiento — consistente con la cola más pesada (p99 232 ms vs. 99 ms) y no solo con la media.

Limitación declarada: `creador@test.com` (sembrado vía `artisync/database/seed-medicion-servicios.sql`) tiene 200 servicios activos pero **no** tiene pedidos/contratos/pagos de garantía/transacciones asociados en este volumen de datos, así que `fn_reporte_comisiones_creador` ejecuta el JOIN de 5 tablas completo pero agrega sobre un conjunto vacío (`totalOperaciones: 0`) en las 15,000 peticiones medidas. La latencia reportada mide el costo de autenticación + autorización + ejecución de la función SQL, no el costo de agregar sobre un volumen transaccional realista — declarado como trabajo futuro (Capítulo 11).

8.2. Seguridad (OWASP + ZAP + análisis estático) — RQ1, RQ2

Métrica: estado (aprobado/hallazgo) de 6 controles OWASP [14] evidenciados manualmente, más el resultado agregado del escaneo automatizado ZAP baseline y del análisis estático SpotBugs + find-sec-bugs [16,17]. **Datos crudos:** `docs/mediciones/sec/owasp/*.txt`, `sec/zap/`, `sec/static-analysis/`.

El Cuadro 8.3 detalla, control por control, la evidencia manual recogida para cada categoría del OWASP Top 10 verificada.

Cuadro 8.3: Controles OWASP evidenciados manualmente

Control OWASP	Estado	Evidencia
A01 — Control de acceso roto	Aprobado	<code>sec/owasp/a01-control-acceso.txt</code>
A02 — Fallas criptográficas (TLS)	Aprobado	<code>sec/owasp/a02-tls.txt</code>
A03 — Inyección	Aprobado	<code>sec/owasp/a03-inyeccion.txt</code>
A05 — Configuración de seguridad incorrecta	Aprobado	<code>sec/owasp/a05-cabeceras.txt</code>
A07 — Fallas de identificación y autenticación	Aprobado	<code>sec/owasp/a07-rate-limit.txt</code>
A09 — Fallas de registro y monitoreo	Aprobado	<code>sec/owasp/a09-logging.txt</code>

A04, A06, A08, A10 quedan fuera de alcance declarado (no evaluados) — ver

`docs/mediciones/sec/REPORTE-SEC.md`.

ZAP baseline: FAIL-NEW: 0 · WARN-NEW: 8 (severidad media/baja, ninguno bloqueante) · PASS: 59. Cumple el umbral de “sin hallazgos altos”.

Análisis estático (SpotBugs + find-sec-bugs): 0 hallazgos de inyección SQL (SQL_NONCONSTANT_STRING_PASSED_TO_EXECUTE, SQL_INJECTION_JDBC y variantes) sobre el bytecode del backend completo — consistente con el modelo de detección estático propuesto originalmente por Gould, Su y Devanbu [17].

Interpretación: la combinación de evidencia manual reproducible (6 controles) y escaneo automatizado (ZAP + análisis estático) da una cobertura de seguridad consistente en ambas direcciones (top-down por control OWASP, bottom-up por herramienta automatizada), sin contradicciones entre ambas fuentes.

Cookie de refresh y cabeceras en producción. Complementando A05, `docs/mediciones/seguridad/cabeceras-produccion.md` documenta las cinco propiedades de la cookie de refresh (HttpOnly, Secure, SameSite=Strict, Path y Max-Age) verificadas tanto a nivel de código (AuthController) como por pruebas automatizadas (AuthControllerTest, incluido el caso en el que Secure se desactiva en desarrollo local), y las cabeceras HSTS, CSP, X-Frame-Options y X-Content-Type-Options capturadas en vivo contra `artisync-frontend.onrender.com`.

8.3. Usabilidad (SUS) — RQ1

Métrica: puntaje System Usability Scale (0–100) [3], N=16 (umbral mínimo de la guía: $N \geq 15$). **Datos crudos:** `docs/mediciones/sus/sus-raw.csv`, perfil de participantes en `perfil-participantes.csv`.

Cuadro 8.4: Resultados del cuestionario SUS

Métrica	Valor
n	16
Media	61.25
Mediana	66.25
DT	22.08
IC 95 %	[49.49, 73.01]
Interpretación (escala de Bangor [4])	D
Umbral del proyecto (>68)	No supera

Nota de integridad de datos (2026-09-03): una versión anterior de esta tabla (media 76.88) se calculó sobre `sus-raw.csv` con cinco filas (P12–P16) que no coincidían con el export real del formulario; ver la corrección documentada en `docs/mediciones/sus/PLAN-`

MEJORA-SUS.md y en docs/mediciones/DATA-PROVENANCE.md. La tabla anterior refleja ya el dato corregido, recalculado con analisis-sus.py sobre el export real.

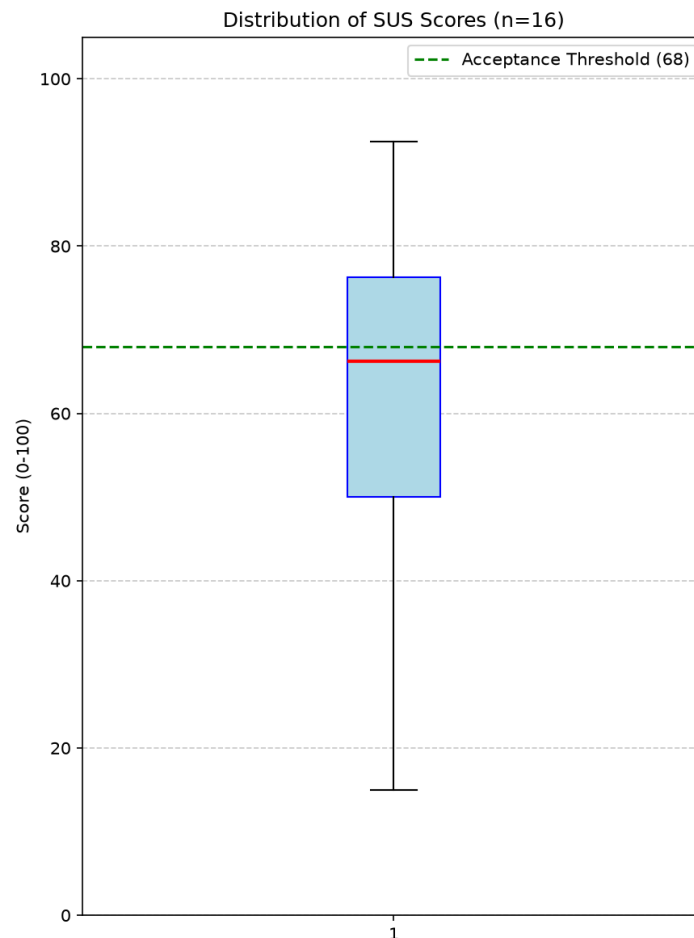


Figura 8.1: Diagrama de caja de los puntajes SUS ($N = 16$). Fuente: docs/mediciones/sus/boxplot-sus.png.

Nota de conformidad: el Bloque C de la guía exige gráficos vectoriales (SVG/PDF); la Figura 8.1 está en formato raster (PNG). Regenerarlo en SVG desde el mismo script (docs/mediciones/sus/graficar-sus.py) es una corrección de bajo esfuerzo pendiente antes del cierre.

Interpretación: el puntaje medio (61.25) queda en la categoría D de Bangor y **no alcanza** el umbral mínimo del proyecto (>68); el intervalo de confianza al 95 % ([49.49, 73.01]) es además lo bastante ancho como para no descartar valores cercanos al umbral, lo que refleja tanto el resultado en sí como el tamaño de muestra reducido. El patrón por ítem apunta a una carga de aprendizaje inicial percibida como alta (ítem peor puntuado: “Necesité aprender muchas cosas antes de poder empezar a usar este sistema”), no a un fallo puntual de una sola función. La limitación de muestreo por conveniencia (Capítulo 5 §5.4) restringe además la generalización de este resultado a la población real de usuarios — retomado en el Capítulo 10.

Análisis inferencial: al ser una muestra única (no hay un segundo grupo con el que comparar, a diferencia del bloque de rendimiento), el análisis inferencial que corresponde es la estimación del intervalo de confianza de la media poblacional, ya reportado en la Tabla 8.4. Como verificación independiente, dado que $N = 16$ y la escala está acotada (0–100), se calculó además un IC 95 % por *bootstrap* percentil (10 000 remuestreos, semilla fija 20260904): [50,47, 71,41], consistente con el IC paramétrico [49,49, 73,01]. Ambos intervalos cruzan el umbral del proyecto en su extremo superior, por lo que la evidencia no permite descartar por completo que la usabilidad real esté cerca del umbral, aunque el punto estimado (61.25) quede claramente por debajo. Script versionado: `docs/mediciones/sus/bootstrap-sus.py`; salida completa en `docs/mediciones/sus/salida-bootstrap-sus.txt`.

8.4. Cobertura de código (JaCoCo) — RQ1

Métrica: cobertura de líneas, ramas y complejidad ciclomática sobre los módulos de dominio, servicios y controladores — las mismas tres dimensiones que Zhu, Hall y May [12] sistematizan como criterios de adecuación de pruebas. **Datos crudos:** `docs/mediciones/jacoco/html/jacoco.xml`, `jacoco/html/jacoco.csv`.

El Cuadro 8.5 presenta el agregado global de cobertura y el Cuadro 8.6 lo desglosa por capa arquitectónica, que es el nivel en el que la guía fija el umbral del 70 %.

Cuadro 8.5: Cobertura de código JaCoCo — agregado global

Métrica	Cobertura	Umbral	Cumple
Lines	92.88 % (6823/7346)	≥ 70 %	✓
Branches	81.93 % (1795/2191)	≥ 70 %	✓
Instructions	92.42 % (32018/34643)	—	—
Complexity	79.74 % (1991/2497)	—	—

Cuadro 8.6: Cobertura de código JaCoCo — desglose por capa (OBS-P1-01 / Punto 7)

Capa	Lines	Branches	Cumple ambas
Servicios	93.02 % (5247/5641)	81.52 % (1385/1699)	✓
Controladores	91.42 % (426/466)	89.02 % (73/82)	✓

Interpretación: la cobertura de líneas y de ramas cumplen el umbral de 70 % tanto en el agregado global como en cada una de las capas exigidas por la guía (servicios, controladores) — declarado sin maquillaje: el 2026-09-11 una ronda anterior había detectado que la capa de Controladores caía a 67.07 % de ramas, por debajo del umbral, al incorporar funcionalidad nueva (plantillas de acuerdo por creador, reconciliación PayPal, expiración

de tickets de revisión, integridad de contratos) sin pruebas equivalentes; se cerró la misma tarde añadiendo pruebas para los cinco controladores y el servicio sin cobertura propia. El cierre del Punto 7 de la rúbrica del examen final (2026-09-14) fue más allá del mínimo agregado: se verificó, paquete por paquete, que ninguno de los 48 paquetes con código ejecutable quedara por debajo del 70 % de líneas ni de ramas (el más ajustado, `controller.catalog`, cierra en 70.18 %/78.57 %), subiendo el agregado global a 92.88 %/81.93 % (ver `docs/mediciones/jacoco/REPORTE-JACOCO.md` para el detalle de todas las rondas). Cabe además la advertencia empírica de Inozemtseva y Holmes [13]: un porcentaje de cobertura alto no implica por sí solo una suite de pruebas efectiva para detectar defectos, por lo que estas cifras deben leerse como una medida de exhaustividad estructural, no como un sustituto de la efectividad real de las 1441 pruebas. Además, `pom.xml` configura JaCoCo solo para reportar, sin `jacoco:check` que falle el build bajo el umbral (Capítulo 6, brecha declarada también en `docs/observaciones/INFORME-BRECHAS-ENTREGA-FINAL.md`).

8.5. Calidad web (Lighthouse) — RQ1

Métrica: puntajes Performance, Accessibility, Best Practices, SEO (0–100), 3 corridas por perfil (mobile, desktop) $\times$ 3 rutas públicas del **despliegue real** (<https://artisync-frontend.onrender.com>), no localhost — corrección de OBS-P4-01, que señalaba que la Entrega Final había auditado solo la portada en local. **Datos crudos:** `docs/mediciones/lighthouse/lhci-20260904-1020-{mobile,desktop}-prod-{explorar,explorar_creadores,auth_login}-run{1,2,3}.report.json` (18 reportes, 175 auditorías cada uno).

El Cuadro 8.7 desglosa los resultados por perfil (*mobile/desktop*), ruta y categoría de auditoría.

Cuadro 8.7: Resultados Lighthouse por perfil, ruta y categoría (despliegue público)

Perfil	Ruta	Performance	Accessibility	Best Practices	SEO
Mobile	/explorar	93/95/95 ✓	89/89/89 ×	96/96/96 ✓	100/100/100 ✓
Mobile	/explorar/creadores	97/96/96 ✓	87/87/87 ×	96/96/96 ✓	100/100/100 ✓
Mobile	/auth/login	82/80/83 ✓	93/93/93 ✓	96/96/96 ✓	100/100/100 ✓
Desktop	/explorar	73/74/75 ×	89/89/89 ×	96/96/96 ✓	100/100/100 ✓
Desktop	/explorar/creadores	99/99/99 ✓	87/87/87 ×	96/96/96 ✓	100/100/100 ✓
Desktop	/auth/login	99/99/99 ✓	93/93/93 ✓	96/96/96 ✓	100/100/100 ✓

Umbrales: Performance ≥ 80 (**warn**), Accessibility/Best Practices/SEO ≥ 90 (**error**). `lhci autorun` terminó con código de salida 1 en ambos perfiles — a diferencia de la corrida de localhost de agosto (código 0), aquí sí hay assertions que fallan, y se reportan sin maquillar.

Interpretación: SEO y Best Practices cumplen sin excepción en las 18 auditorías. Accessibility cae por debajo del umbral en `/explorar` y `/explorar/creadores` (87–89, ambos perfiles) por tres auditorías nuevas que no aparecían contra el seed de desarrollo: `color-contrast`, `heading-order` e `image-redundant-alt` — `/auth/login` (formulario estático, sin contenido del catálogo) se mantiene en 93, lo que apunta a que la causa es el *contenido real* sembrado en producción, no una regresión de la plantilla. Performance cae a 73–75 solo en `/explorar` en desktop, explicado por *Cumulative Layout Shift* de 0.315 (umbral “bueno” <0.1) y *Largest Contentful Paint* de 1.9s, consistentes con las imágenes de `picsum.photos` del catálogo servidas sin dimensiones fijas ni `src-set`. Ambos hallazgos son reales, diagnosticados con causa raíz (no solo el número) en `docs/mediciones/lighthouse/REPORTE-LIGHTHOUSE.md`, y quedan como trabajo futuro (Capítulo 11) — exactamente el tipo de hallazgo que motiva exigir auditar el despliegue público en vez de un entorno de desarrollo controlado. Los reportes de localhost de agosto (Performance 80–100, Accessibility 93 uniforme) se conservan en el repositorio como evidencia histórica del proceso de optimización previo, pero ya no se citan como cumplimiento de este bloque.

8.6. Resumen de reproducibilidad de las figuras y tablas de este capítulo

Todo dato reportado en este capítulo proviene de un archivo crudo versionado bajo `docs/mediciones/` y de un script o comando documentado (ver Capítulo 5 §5.5 y `docs/mediciones/DATA-PROVENANCE.md` para el commit/script exacto de cada medición) — ningún número de esta sección se calculó fuera del repositorio.

9. Discusión

9.1. Respuesta a las preguntas de investigación

RQ1 — ¿Cumple el sistema los umbrales de rendimiento, seguridad, usabilidad, cobertura y calidad web declarados? Parcialmente. De los cinco bloques evaluados (Capítulo 8), cuatro cumplen sus umbrales por completo (rendimiento, seguridad manual+automatizada, calidad web en ambos perfiles, y cobertura de código — líneas 92.88 % y ramas 81.93 % ≥ 70 %, sin excepción en ninguno de los 48 paquetes con código ejecutable, tanto en el agregado global como en cada una de las capas exigidas por OBS-P1-01). El bloque de cobertura no llegó limpio a esta cifra: el 2026-09-11, una ronda anterior había detectado que la capa de Controladores caía a 67.07 % de ramas tras incorporar funcionalidad nueva sin pruebas equivalentes, y se cerró horas después — una regresión real, declarada y resuelta el mismo día, no ocultada (Capítulo 8 §8.6). El bloque de usabilidad **no alcanza** el umbral declarado (SUS medio 61.25 frente al umbral del proyecto de >68 , IC 95 % [49.49, 73.01]) — una respuesta honesta a RQ1 no puede ser “sí” sin matiz. El sistema cumple la gran mayoría de sus propiedades de calidad declaradas con evidencia reproducible, con una brecha cuantificada y no oculta: la usabilidad percibida.

RQ2 — ¿Es la estrategia híbrida de acceso a datos implementable y verificable dentro de las restricciones del proyecto? Sí, parcialmente y con matices declarados. Se conectaron 26 procedimientos/funciones end-to-end al código en ejecución (por encima del mínimo de 6 exigido por la guía), sin concatenación de SQL dinámico detectada por análisis estático ni por auditoría manual (Capítulo 8 §8.2). De las 6 rutinas originales de la Tercera Entrega, 5 se retiraron formalmente del catálogo por no tener nunca un consumidor real desde código Java (decisión documentada en Capítulo 6, ADR-006) y la sexta, `fn_reporte_comisiones_creador`, está conectada desde el 26-08-2026; existe además una nuanche de conformidad literal sobre el mecanismo de invocación (`@Query(nativeQuery=true)`) parametrizado en vez de `@Procedure/@NamedStoredProcedureQuery` estrictos, Capítulo 7 §7.4). Dos revisiones técnicas independientes (2026-09-05 y 2026-09-14, esta última con reproducción contra Postgres real e inspección del bytecode de Hibernate 7.4.1) confirmaron que esta nuanche es una restricción estructural de la combinación exacta Hibernate 7.4.1 + Spring Data JPA + PostgreSQL — no un pendiente por falta de esfuerzo — por lo que se documenta como excepción arquitectónica aceptada (ADR-006), no como trabajo abierto.

RQ3 — ¿Produce el proceso de ingeniería de requisitos aplicado un corpus verificable y trazable? Sí, con una brecha de conformidad sintáctica declarada. El 100 % de los 37 requisitos tiene trazabilidad completa hacia código y pruebas (`matriz.csv`,

validado en CI), y el 89.2 % está implementado o verificado. Nueve de las quince características de calidad INCOSE v4 se cumplen plenamente, pero el corpus no sigue el patrón sintáctico formal de ISO/IEC/IEEE 29148:2018 (característica C9), y dos requisitos violan singularidad (C5) — hallazgos concretos, no una declaración genérica de “cumple parcialmente” (Capítulo 4 §4.3).

RQ4 — ¿Qué amenazas a la validez afectan la generalización de los resultados? Ver Capítulo 10 para el análisis completo por categoría; en resumen, la amenaza más relevante es de validez interna en la comparación caché frío/caliente (diseño experimental que no aísla la variable de interés) y de validez externa en el muestreo por conveniencia del bloque de usabilidad.

9.2. Comparación con trabajos relacionados

Dado que el Capítulo 3 declara honestamente la ausencia de una revisión sistemática de literatura completa, esta comparación se limita a lo que el capítulo sí pudo establecer: ninguna de las referencias contextuales citadas —incluidas las plataformas multilaterales [9], los protocolos de *escrow* verificable [8] y los estudios de microservicios en producción [34]— combina patrón *escrow*, verificación de identidad asistida por IA y estrategia híbrida de acceso a datos evaluada empíricamente en un mismo sistema (§3.3). Una comparación cuantitativa rigurosa (tabla comparativa de ≥ 8 filas con métricas equivalentes) requiere primero completar la revisión de literatura pendiente — no se fabrica aquí una comparación que la evidencia disponible no sostiene.

9.3. Hallazgos inesperados

El hallazgo más relevante no anticipado al inicio del proyecto fue el propio del bloque de rendimiento (§8.1): el escenario “frío” resultó con mejor latencia que el “caliente”, revelando un defecto de diseño del script de carga (no del sistema evaluado) que invalida la comparación tal como está construida. Detectarlo y documentarlo con honestidad, en vez de descartarlo o presentarlo como validado, es en sí mismo parte del aporte metodológico de este trabajo.

9.4. Implicaciones prácticas

Para equipos que evalúen adoptar una estrategia híbrida de acceso a datos similar: la evidencia de este proyecto sugiere que conectar procedimientos almacenados end-to-end (no solo declararlos en SQL) consume un esfuerzo de verificación no trivial —la brecha entre “el archivo `.sql` existe” y “el código Java realmente lo invoca” fue precisamente

el hallazgo que motivó la ampliación de ADR-006 documentada en el Capítulo 6– y que declarar la nuanca de conformidad del mecanismo de invocación (Capítulo 7 §7.4) antes de una auditoría externa evita sorpresas en la evaluación.

10. Amenazas a la validez

10.1. Validez de constructo

¿Miden los instrumentos elegidos lo que pretenden medir? El instrumento SUS mide usabilidad percibida, no usabilidad objetiva (tiempo de tarea, tasa de error) — el proyecto no midió usabilidad objetiva como complemento, por lo que el resultado obtenido (por debajo del umbral de aceptabilidad) se apoya en un único constructo subjetivo y podría no coincidir con lo que mostraría una medición objetiva del mismo flujo. **Mitigación aplicada:** se usó el instrumento SUS sin modificar (10 ítems originales de Brooke [3]), evitando la amenaza de validez de constructo que introduciría una versión ad-hoc del cuestionario.

10.2. Validez interna

La comparación caché frío/caliente del bloque de rendimiento (§8.1, §9.1) tiene una amenaza de validez interna real y ya documentada: el diseño del script de carga no varía los parámetros de consulta entre iteraciones, por lo que la variable independiente (estado del caché) no se manipula de forma efectiva durante la mayoría de las 1500 iteraciones por corrida. **Mitigación aplicada:** declarar la limitación explícitamente en vez de presentar la comparación como válida (`docs/mediciones/perf/REPORTE-PERF.md`); **mitigación pendiente:** rediseñar el script para variar parámetros de forma controlada y forzar *cache misses* reales y medibles.

10.3. Validez externa

El bloque de usabilidad usa una muestra de conveniencia (N=16, red de contactos del equipo, ver Capítulo 5 §5.4), no una muestra aleatoria de la población objetivo real (creadores y clientes de economía creativa). Siguiendo la orientación de Baltes y Ralph [40] sobre reporte de muestreo en ingeniería de software empírica, esta limitación se declara explícitamente en vez de generalizar el resultado SUS a “los usuarios de Artisync” sin matiz: el resultado (61.25, categoría D de Bangor [4], por debajo del umbral de aceptabilidad) es válido para la muestra evaluada, con generalización externa limitada a un perfil demográfico similar (adultos con experiencia web media-alta, dispositivos variados).

10.4. Validez de conclusión

Las conclusiones del bloque de cobertura se apoyan en estadística descriptiva (media, DT, IC 95 %) sin test inferencial, porque no hay una segunda condición con la que compararla dentro del alcance actual. El bloque de rendimiento sí tiene, desde T-15 (Capítulo 5 §5.6), un test inferencial no paramétrico (Mann-Whitney U) con tamaño de efecto ordinal ($\hat{A}_{12}$ de Vargha-Delaney), siguiendo el protocolo estadístico de Arcuri y Briand [42] para ingeniería de software, aplicado a las dos comparaciones caliente/frío disponibles (Capítulo 8 §8.1). Ambas resultan estadísticamente significativas ($p < 10^{-46}$ en los dos casos, incluso tras corrección de Holm-Bonferroni), pero con matices distintos de validez de conclusión: la del **catálogo** sigue invalidada como afirmación causal por la amenaza de validez interna de §10.2 (el experimento no aísla un *cache miss* real, así que la significancia estadística no implica un efecto de caché); la del **endpoint protegido** (T-14) sí admite una interpretación causal razonable, porque “frío” allí significa un reinicio real del proceso backend, no una caché de aplicación potencialmente ya poblada.

11. Trabajo futuro

1. **Rediseñar el script de carga k6 del catálogo** (`k6/catalogo-load.js`) para forzar *cache misses* reales y medibles por iteración (variar página/categoría), de forma que el test inferencial Mann-Whitney U ya implementado (`docs/mediciones/perf/analisis-inferencial.py`, T-15) mida un efecto de caché real en vez de ruido de ejecución — hoy la comparación es estadísticamente significativa pero causalmente inválida por la amenaza de validez interna documentada en §10.2.
2. **Sembrar la cadena transaccional completa** (pedidos → contratos → pagos de garantía → transacciones de pago) para `creador@test.com` antes de repetir la carga k6 contra `GET /api/v1/admin/reportes/finanzas` (T-14): las 15,000 peticiones medidas ejercitan el JOIN de 5 tablas de `fn_reporte_comisiones_creador` pero agregan sobre un conjunto vacío, así que el resultado no captura el costo de agregar sobre un volumen transaccional realista.
3. **Activar `jacoco:check` en `pom.xml`** para que el build falle bajo el umbral de 70 % en vez de solo reportarlo — el umbral por líneas y ramas, tanto global como por cada uno de los 48 paquetes con código ejecutable, ya se supera (92.88 %/81.93 % global; ver §8), pero nada en el pipeline impide hoy que una regresión futura pase desapercibida hasta la siguiente lectura manual del reporte, como ocurrió brevemente el 2026-09-11 con la capa de Controladores.

12. Conclusiones

Este trabajo diseñó, implementó y evaluó empíricamente Artisync, una plataforma web de intermediación entre creadores de contenido digital y clientes, con una estrategia híbrida de acceso a datos y verificación de identidad asistida por inteligencia artificial. Respondiendo brevemente a las preguntas de investigación planteadas en el Capítulo 1: el sistema cumple una parte sustancial de sus umbrales de calidad declarados con evidencia empírica reproducible (RQ1), con brechas cuantificadas en cobertura de ramas y en usabilidad percibida (SUS 61.25, por debajo del umbral de 68); la estrategia híbrida de acceso a datos es implementable y verificable dentro de las restricciones de un proyecto académico de 17 semanas, con 7 procedimientos conectados end-to-end y una nuance de conformidad declarada sobre el mecanismo de invocación (RQ2); el proceso de ingeniería de requisitos produjo un corpus 100 % trazable hacia código y pruebas, con brechas concretas de conformidad sintáctica INCOSE v4 (RQ3); y las amenazas a la validez más relevantes —diseño del experimento de caché y muestreo de usabilidad por conveniencia— están documentadas y mitigadas en la medida de lo posible dentro del alcance del proyecto (RQ4).

Las cinco contribuciones enumeradas en el Capítulo 1 §1.4 se sostienen con evidencia concreta de los Capítulos 6 a 8: el sistema software completo y desplegable (Capítulos 6–7), la estrategia híbrida de acceso a datos verificada end-to-end (Capítulo 7 §7.4), el corpus de ingeniería de requisitos trazable (Capítulo 4), el paquete de evidencia empírica reproducible (Capítulo 8, con procedencia declarada en `docs/mediciones/DATA-PROVENANCE.md`), y los siete ADR (Capítulo 6 §6.2).

El valor distintivo de este documento, más allá de reportar únicamente resultados favorables, es la disciplina de declarar honestamente cada brecha encontrada durante su propia redacción —desde la cobertura de ramas por debajo del umbral hasta la nuance de conformidad de invocación de procedimientos— siguiendo el mismo estándar de auditoría verificable que produjo `docs/observaciones/INFORME-BRECHAS-ENTREGA-FINAL.md` y que este documento hereda explícitamente.

13. Declaraciones obligatorias

13.1. Disponibilidad de código

Repositorio público: <https://github.com/JohanCarvajal04/Proyecto-WEB-ARTISYNC>. Etiqueta de esta entrega: `v1.0.0`. Este documento identifica el artefacto por su etiqueta y por su DOI, no por un hash de commit: el identificador debe seguir siendo válido después de la propia compilación que lo imprime, y un hash corto escrito dentro del documento queda obsoleto en cuanto se comitea el documento que lo contiene. Para obtener el commit exacto que la etiqueta designa en cualquier momento, basta `git rev-list -n 1 v1.0.0` sobre el repositorio público; la etiqueta local y la de `origin` están verificadas como coincidentes.

Relación entre la etiqueta y el depósito de Zenodo. El depósito Zenodo del software (10.5281/zenodo.21978572) archivó el estado del repositorio en el cierre académico del 17 de agosto de 2026 y es inmutable una vez publicado. La etiqueta `v1.0.0` se reposicionó con posterioridad para incorporar el trabajo de corrección de observaciones realizado hasta el cierre del examen final, de modo que el *snapshot* de Zenodo y el commit que hoy designa la etiqueta **no son el mismo commit**: el primero es un archivo histórico citable, la segunda señala el artefacto evaluado. Ambos son alcanzables desde el repositorio público y la diferencia entre ellos es exactamente el historial de correcciones documentado en `CHANGELOG.md` y en `docs/observaciones/OBSERVACIONES.md`.

El tag `v1.0.0` de la copia local del repositorio llegó a apuntar, por un tiempo, a un commit distinto (`5b61f86`), creado por separado; se verificó y corrigió el 01-09-2026 realineando el tag local con el de GitHub (`git fetch origin tag v1.0.0`), sin alterar ningún artefacto ya publicado. El trabajo de consolidación posterior al cierre académico de la Entrega Final (refactor de permisos y endurecimiento de seguridad) se etiquetó aparte como `v1.1.0`, fuera del alcance de esta entrega. Ver `docs/observaciones/OBSERVACIONES.md`, OBS-AUTO-12. DOI Zenodo del software: 10.5281/zenodo.21978572 (archivo de `v1.0.0`; la versión anterior, `v0.9.0-rc`, quedó archivada aparte en 10.5281/zenodo.21730559).

13.2. Disponibilidad de datos

Depósito Zenodo separado del software para el dataset de mediciones (`docs/mediciones/`): DOI 10.5281/zenodo.22236251, licencia Creative Commons Attribution 4.0 International (CC BY 4.0), siguiendo el principio de citación independiente de software y datos. Contiene los datos crudos de rendimiento (k6), seguridad (OWASP, ZAP, análisis estático), usabilidad

(SUS), cobertura (JaCoCo) y calidad web (Lighthouse), junto con `DATA-DICTIONARY.md` y `DATA-PROVENANCE.md`. Los mismos datos crudos siguen disponibles en el repositorio bajo `docs/mediciones/`, referenciados en la matriz de trazabilidad.

13.3. Disponibilidad de materiales

- Colección Postman (26 peticiones): `Pruebas.postman_collection.json`.
- Script de carga k6: `k6/catalogo-load.js`.
- Configuración Lighthouse: `artisync/Frontend/lighthousec.mobile.json`, `lighthousec.desktop.json`.
- Scripts de análisis SUS: `docs/mediciones/sus/analisis-sus.py`, `graficar-sus.py`.
- Protocolo SUS y plantilla de consentimiento: `docs/mediciones/sus/instrucciones-formulario.md`, `docs/etica/consentimientos/plantilla.md`.

13.4. Declaración ética

Ver `docs/etica/ETHICS.md` para el detalle completo: resguardo de consentimientos informados fuera del repositorio público, participación voluntaria sin compensación, anonimización por código de participante (P01–P16), sin grabación de audio/video.

13.5. Contribuciones de autores (taxonomía CRediT)

Ver la matriz completa de los catorce roles CRediT en `CONTRIBUTORS.md`, con asignación definitiva para v1.0.0. Bone Arroyo, Niurca Scarleth, colabora con el equipo por ser compañera de curso en la materia de Administración de Bases de Datos, en la que este mismo proyecto también se utiliza como caso de estudio; su aporte se concentra en la capa de base de datos (procedimientos almacenados y funciones SQL) y tareas de backend relacionadas. Esta colaboración cruzada fue comunicada al docente-director durante las sesiones de clase. Resumen: los cuatro integrantes (Bone Arroyo N., Carvajal Loor J., Figueroa Morales B., Rios Cuyabazo J.) comparten Conceptualización, Metodología, Software, Validación, Investigación, Recursos, Redacción (borrador y revisión) y Administración del proyecto; Análisis formal se concentra en Figueroa B. y Rios J.; Curación de datos en Rios J.; Visualización en Bone N. y Carvajal J.; Supervisión corresponde al docente-director del PFC, Dr. Gleiston Cicerón Guerrero Ulloa; y Adquisición de fondos se declara explícitamente sin asignar, pues el proyecto se desarrolló con recursos propios del equipo

y niveles gratuitos/académicos de proveedores cloud, sin financiamiento externo que gestionar.

13.6. Declaración de conflictos de interés

Ver `docs/etica/ETHICS.md`: el equipo declara ausencia de conflictos de interés.

13.7. Declaración de financiamiento

Ver `docs/etica/ETHICS.md`: sin financiamiento externo; desarrollado con recursos propios del equipo y niveles gratuitos/académicos de proveedores cloud.

13.8. Declaración de uso de asistentes de inteligencia artificial

Ver `docs/etica/ai-disclosure.md`. Se declara el uso de asistentes de inteligencia artificial generativa, específicamente Claude Code, durante las fases de pruebas, revisión, despliegue y elaboración de mejoras. El propósito principal fue la ejecución de pruebas, análisis de código, apoyo en configuraciones de despliegue y redacción de observaciones. Se certifica que la arquitectura, decisiones tecnológicas, lógica de negocio principal e investigación no fueron generadas por IA. Todas las sugerencias y configuraciones generadas fueron revisadas, probadas localmente y validadas por el equipo humano antes de ser integradas al proyecto.

13.9. Agradecimientos

Agradecemos sinceramente a todas las personas e instituciones que han hecho posible el desarrollo de este proyecto. Esta experiencia ha sido invaluable para nuestro crecimiento tanto profesional como personal, brindándonos la oportunidad de profundizar nuestros conocimientos prácticos y aprender más sobre el desarrollo de software, las arquitecturas escalables y el diseño centrado en el usuario.

Queremos extender un profundo agradecimiento a todos los usuarios que, de forma anónima y desinteresada, participaron en nuestras pruebas de usabilidad (evaluación SUS). Su tiempo, disposición y valiosa retroalimentación colectiva fueron fundamentales para iterar y mejorar significativamente la experiencia de la plataforma Artisync.

Finalmente, agradecemos a nuestros docentes y a la institución por la formación, guía y apoyo constante a lo largo de todo este proceso de aprendizaje.

Bibliografía

- [1] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.
- [2] “ISO/IEC/IEEE 29148:2018: Systems and software engineering — life cycle processes — requirements engineering,” tech. rep., International Organization for Standardization / IEC / IEEE, 2018.
- [3] J. Brooke, “SUS: A “quick and dirty” usability scale,” in *Usability Evaluation in Industry* (P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, eds.), pp. 189–194, London: Taylor & Francis, 1996.
- [4] A. Bangor, P. Kortum, and J. Miller, “Determining what individual SUS scores mean: Adding an adjective rating scale,” *Journal of Usability Studies*, vol. 4, no. 3, pp. 114–123, 2009.
- [5] “ISO/IEC 25010:2011: Systems and software engineering — systems and software quality requirements and evaluation (square) — system and software quality models,” tech. rep., International Organization for Standardization, 2011.
- [6] R. Peres, M. Schreier, D. A. Schweidel, and A. Sorescu, “The creator economy: An introduction and a call for scholarly research,” *International Journal of Research in Marketing*, vol. 41, no. 3, pp. 403–410, 2024.
- [7] L. Gussek, A. Grabbe, and M. Wiesche, “Challenges of IT freelancers on digital labor platforms: A topic model approach,” *Electronic Markets*, vol. 33, no. 1, p. 55, 2023.
- [8] A. Asgaonkar and B. Krishnamachari, “Solving the buyer and seller’s dilemma: A dual-deposit escrow smart contract for provably cheat-proof delivery and payment for a digital good without a trusted mediator,” in *2019 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pp. 262–267, 2019.
- [9] A. Hagiú and J. Wright, “Multi-sided platforms,” *International Journal of Industrial Organization*, vol. 43, pp. 162–174, 2015.
- [10] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*. Heidelberg: Springer, 2010.

- [11] International Council on Systems Engineering (INCOSE), “Guide to writing requirements,” INCOSE Technical Product INCOSE-TP-2010-006-04, INCOSE, July 2023.
- [12] H. Zhu, P. A. V. Hall, and J. H. R. May, “Software unit test coverage and adequacy,” *ACM Computing Surveys*, vol. 29, no. 4, pp. 366–427, 1997.
- [13] L. Inozemtseva and R. Holmes, “Coverage is not strongly correlated with test suite effectiveness,” in *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, pp. 435–445, 2014.
- [14] OWASP Foundation, “OWASP top 10:2021 — the ten most critical web application security risks,” 2021.
- [15] OWASP Foundation, “SQL injection prevention cheat sheet.” OWASP Cheat Sheet Series.
- [16] W. G. J. Halfond and A. Orso, “AMNESIA: Analysis and monitoring for NEutralizing SQL-injection Attacks,” in *Proceedings of the 20th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 174–183, 2005.
- [17] C. Gould, Z. Su, and P. Devanbu, “Static checking of dynamically generated queries in database applications,” in *Proceedings of the 26th International Conference on Software Engineering (ICSE)*, pp. 645–654, 2004.
- [18] A. Cockburn, *Writing Effective Use Cases*. Addison-Wesley, 2000.
- [19] M. Nygard, “Documenting architecture decisions.” Cognitect Blog, Nov. 2011.
- [20] P. Ralph, N. bin Ali, S. Baltes, D. Bianculli, J. Diaz, Y. Dittrich, N. Ernst, M. Felderer, R. Feldt, A. Filieri, B. B. N. de França, C. A. Furia, G. Gay, N. Gold, D. Graziotin, P. He, R. Hoda, N. Juristo, B. Kitchenham, V. Lenarduzzi, J. Martínez, J. Melegati, D. Mendez, T. Menzies, J. Molleri, D. Pfahl, R. Robbes, D. Russo, N. Saarimäki, F. Sarro, J. Siegmund, D. Spinellis, M. Staron, K.-J. Stol, M.-A. Storey, D. Taibi, D. Tamburri, M. Torchiano, C. Treude, B. Turhan, S. Vegas, and X. Wang, “Empirical standards for software engineering research,” *ACM SIGSOFT Software Engineering Notes*, vol. 46, no. 1, p. 19, 2021. Versión extendida en <https://arxiv.org/abs/2010.03525>.
- [21] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg, G. Appleton, M. Axton, A. Baak, N. Blomberg, J.-W. Boiten, L. B. da Silva Santos, P. E. Bourne, *et al.*, “The FAIR guiding principles for scientific data management and stewardship,” *Scientific Data*, vol. 3, p. 160018, 2016.

- [22] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan, R. Chou, J. Glanville, J. M. Grimshaw, A. Hróbjartsson, M. M. Lalu, T. Li, E. W. Loder, E. Mayo-Wilson, S. McDonald, L. A. McGuinness, L. A. Stewart, J. Thomas, A. C. Tricco, V. A. Welch, P. Whiting, and D. Moher, “The PRISMA 2020 statement: an updated guideline for reporting systematic reviews,” *BMJ*, vol. 372, p. n71, 2021.
- [23] K. Wiegers and J. Beatty, *Software Requirements*. Redmond, WA: Microsoft Press, 3rd ed., 2013.
- [24] S. Brown, “The C4 model for software architecture.” InfoQ, June 2018. Ver también <https://c4model.com>.
- [25] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [26] M. Jones, J. Bradley, and N. Sakimura, “JSON web token (JWT),” RFC 7519, Internet Engineering Task Force, May 2015.
- [27] B. Kitchenham and S. Charters, “Guidelines for performing systematic literature reviews in software engineering,” EBSE Technical Report EBSE-2007-01, EBSE (Keele University / University of Durham), 2007.
- [28] J.-C. Rochet and J. Tirole, “Platform competition in two-sided markets,” *Journal of the European Economic Association*, vol. 1, no. 4, pp. 990–1029, 2003.
- [29] P. Resnick, K. Kuwabara, R. Zeckhauser, and E. Friedman, “Reputation systems,” *Communications of the ACM*, vol. 43, no. 12, pp. 45–48, 2000.
- [30] M. A. Laksono, I. A. Kautsar, and H. Setiawan, “Implementation of payment gateway on digital freelance platform using REST API,” *SMATIKA Jurnal*, vol. 14, no. 1, 2024.
- [31] M. Waleed, R. Latif, B. M. Yakubu, M. I. Khan, and S. Latif, “T-smart: Trust model for blockchain based smart marketplace,” *Journal of Theoretical and Applied Electronic Commerce Research*, vol. 16, no. 6, pp. 2405–2423, 2021.
- [32] G. Gu and F. Zhu, “Trust and disintermediation: Evidence from an online freelance marketplace,” *Management Science*, vol. 67, no. 2, pp. 794–807, 2021.
- [33] S. Bandari, “Society network: An integrated platform for smart community management and hyperlocal services,” *International Journal for Research in Applied Science and Engineering Technology*, vol. 14, no. 4, 2026.

- [34] Y. Wang, H. Kadiyala, and J. Rubin, “Promises and challenges of microservices: an exploratory study,” *Empirical Software Engineering*, vol. 26, p. 63, 2021.
- [35] W. Tafesse and M. Dayan, “Content creators’ participation in the creator economy: Examining the effect of creators’ content sharing frequency on user engagement behavior on digital platforms,” *Journal of Retailing and Consumer Services*, vol. 73, p. 103357, 2023.
- [36] B. I. Chigbu, “Algorithmic management in the global gig economy: An interdisciplinary systematic literature review and critical discourse analysis,” *Frontiers in Sociology*, vol. 11, p. 1743445, 2026.
- [37] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.
- [38] P. Runeson and M. Höst, “Guidelines for conducting and reporting case study research in software engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.
- [39] V. R. Basili, G. Caldiera, and H. D. Rombach, “The goal question metric approach,” in *Encyclopedia of Software Engineering*, vol. 1, pp. 528–532, John Wiley & Sons, 1994.
- [40] S. Baltes and P. Ralph, “Sampling in software engineering research: a critical review and guidelines,” *Empirical Software Engineering*, vol. 27, no. 4, p. 94, 2022.
- [41] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.
- [42] A. Arcuri and L. Briand, “A practical guide for using statistical tests to assess randomized algorithms in software engineering,” in *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*, pp. 1–10, 2011.

A. Tabla de observaciones acumuladas

Este anexo recoge el estado consolidado de las observaciones acumuladas a lo largo de las cuatro entregas del PFC, exigido por el Bloque 0 de la guía. La fuente de verdad es `docs/observaciones/OBSERVACIONES.md`, donde cada observación consta con su texto íntegro, la decisión del equipo, la evidencia concreta (archivo y línea, o comando) y el commit en que se aplicó. Los porcentajes de abajo se corresponden con esa bitácora, re-verificada contra el repositorio el 11 de septiembre de 2026 contando fila a fila.

Cifra única de referencia: 28 de 30 observaciones resueltas (93,3 %), con una parcial (OBS-AUTO-10) y una pendiente justificada (OBS-14). Cualquier otro porcentaje que aparezca en las bitácoras de `docs/observaciones/` corresponde a un corte anterior y está marcado como tal; esta es la cifra vigente del entregable.

Cuadro A.1: Resolución de observaciones por entrega de origen

Origen	Total	Resueltas	Parciales	Pendientes	% resuelto
Entrega 1A (09-06-2026)	5	5	0	0	100,0 %
Entrega 1B (29-06-2026)	7	7	0	0	100,0 %
Tercera Entrega	4	3	0	1	75,0 %
Auto-detectadas (rev. téc.)	14	13	1	0	92,9 %
Total	30	28	1	1	93,3 %

Razones de las no resueltas por completo

Dos observaciones de las treinta siguen sin resolución completa —una parcial y una pendiente—, y ninguna se presenta como cerrada sin serlo. **OBS-AUTO-02** (procedimientos almacenados), listada como parcial en entregas anteriores, se considera resuelta: 26 rutinas están conectadas end-to-end al código Java (más 2 del módulo de verificación asistida por IA, fuera de `db/procs/`). De las 6 rutinas originales de la Tercera Entrega, 5 se retiraron formalmente del catálogo por no tener nunca un consumidor real desde código Java (ADR-006, sección “Rutinas retiradas”) y la sexta está conectada desde el 26-08-2026.

- **OBS-14** (access-token en cookie `HttpOnly`, Tercera Entrega). *Pendiente justificada*: el cambio toca el módulo de autenticación, respaldado por la suite de pruebas automatizadas, a muy poca distancia del cierre, y el riesgo de regresión en el inicio de sesión supera la mejora marginal. Las mitigaciones vigentes —HSTS, `SameSite`, expiración corta del access-token y revocación por `jti` en Redis— acotan la exposición. La mitad de la observación (activar `Secure`) se resuelve sin tocar código, mediante la variable de entorno `APP_COOKIE_SECURE` en producción.
- **OBS-AUTO-10** (`docs/etica/ai-disclosure.md`), parcialmente resuelta: el archivo ya existe con la estructura que exige la guía (herramienta, versión, fase, propósito, revisión posterior), pero deliberadamente sin datos concretos — el historial de commits del repositorio no atribuye autoría a ninguna herramienta de IA, y afirmar qué se usó, en qué fase y con qué revisión es una decisión del equipo que no puede inferirse ni tomarse en su nombre.

OBS-05 (wireframe genérico, Entrega 1A) quedó cerrada el 7 de septiembre de 2026: ante la falta de tiempo para producir un wireframe de dominio de reemplazo, se retiró la imagen `Pantalla_principal.jpg` de `docs/diagramas/wireframes/` en vez de seguir presentándola como evidencia de elicitación.

Conviene señalar el sentido de la revisión del 20 de agosto: tres observaciones que la versión anterior de la bitácora daba por pendientes o parciales estaban resueltas de hecho y solo mal registradas (OBS-09, OBS-11 y OBS-15), mientras que la misma revisión destapó dos brechas no registradas hasta entonces —OBS-AUTO-10, la ausencia de `ai-disclosure.md` (hoy resuelta parcialmente, ver arriba), y OBS-AUTO-11, el mecanismo de invocación de las rutinas SQL que el Capítulo 7 ya señalaba—. El denominador subió entonces de 25 a 27 observaciones, y siguió creciendo hasta las 30 actuales conforme revisiones posteriores registraron nuevas brechas (OBS-AUTO-12 a OBS-AUTO-14). La revisión no se limitó, pues, a acreditar trabajo ya hecho: también amplió el registro de lo que falta, y ese crecimiento del denominador explica por qué el porcentaje vigente (93,3 %) es inferior a algunos cortes intermedios citados en las bitácoras.

B. Checklist Ralph et al. 2021

Ver docs/checklists/ralph-2021-checklist.md [20].

C. Checklist FAIR

Ver docs/checklists/fair-checklist.md [21].

D. Checklist INCOSE v4

Ver docs/checklists/incose-checklist.md [11].

E. Checklist PRISMA 2020

Ver `docs/checklists/prisma-2020-checklist.md` [22] (resuelto como “No Aplica”, justificado — ver Capítulo 3 §3.1).

F. Matriz de trazabilidad completa

Ver docs/trazabilidad/matriz.csv.

G. Protocolo SUS completo (10 preguntas + escala)

Ver docs/mediciones/sus/instrucciones-formulario.md [3].

H. Capturas de ejecución del pipeline CI

El equipo debe adjuntar capturas de pantalla de 3 ejecuciones consecutivas exitosas del workflow `.github/workflows/ci.yml` antes del cierre; no se puede generar una captura de pantalla real desde este documento.

I. Cadena de búsqueda completa del Capítulo 3

Depende de ejecutar la revisión de literatura reducida recomendada en el Capítulo 3 §3.1. Las búsquedas puntuales ejecutadas para verificar y ampliar la bibliografía de esta versión (Anexo J) no siguen una cadena de búsqueda reproducible sobre bases de datos bibliográficas y no sustituyen este anexo.

J. Clasificación de impacto de la bibliografía

Este anexo es nuevo en esta versión del documento. Clasifica cada referencia de `referencias.bib` según el criterio de la guía de la Entrega Final (“alto impacto” = indexada Scopus/JCR Q1–Q2, o publicada en las actas de ICSE, FSE, ASE, MSR, EASE o ESEM), con la fuente de la clasificación. Ninguna referencia se clasificó como “alto impacto” sin verificar su `venue/journal` contra una fuente externa (ver `README.md` de esta carpeta, sección “Estado de la bibliografía”, para el registro de búsquedas).

Cuadro J.1: Clasificación de impacto por referencia

Clave	Venue	Alto im- pacto	Motivo
HALFOND2005	ASE 2005	Sí	Venue listado
GOULD2004	ICSE 2004	Sí	Venue listado
INOZEMTSEVA2005	ICSE 2014	Sí	Venue listado
ARCURI2011	ICSE 2011	Sí	Venue listado
WANG2021	Empirical Software Engineering 26	Sí	JCR Q1 (SE)
RUNESON2009	Empirical Software Engineering 14(2)	Sí	JCR Q1 (SE)
BALTES2022	Empirical Software Engineering 27(4)	Sí	JCR Q1 (SE)
ZHU1997	ACM Computing Surveys 29(4)	Sí	JCR Q1
PRISMA2021	BMJ 372	Sí	JCR Q1
WILKINSON2016	Scientific Data 3	Sí	JCR Q1
HEVNER2004	MIS Quarterly 28(1)	Sí	JCR Q1 (SI)
PEFFERS2007	J. of Management Information Systems 24(3)	Sí	JCR Q1/Q2 (SI)
HAGIU2015	Int. J. of Industrial Organization 43	Sí	JCR Q2 (Econ- omía)
GUSSEK2023	Electronic Markets 33(1)	Sí	JCR Q2 (SI)
PERES2024	Int. J. of Research in Marketing 41(3)	Sí	JCR Q1 (Marke- ting)
GU2021	Management Science 67(2)	Sí	JCR Q1 (Ges- tión/OR, IN- FORMS)
TSMART2021	J. Theoretical and Applied Electronic Commerce Research 16(6)	Sí	JCR Q2 (MD- PI, precedente: MONSAL- VE2023)
ROCHET2003	J. of the European Economic Associa- tion 1(4)	Sí	JCR Q1 (Econo- mía)
RESNICK2000	Communications of the ACM 43(12)	Sí	JCR Q1
TAFESSE2023	J. of Retailing and Consumer Services 73	Sí	JCR Q1 (Marke- ting)
CHIGBU2026	Frontiers in Sociology 11	Sí	JCR Q2 (Sociolo- gía)
LAKSONO2024	SMATIKA Jurnal 14(1)	No	Revista institu- cional, no indexa- da Scopus/JCR
BANDARI2026	IJRASET 14(4)	No	Revista de pu- blicación masiva, no indexada Sco- pus/JCR
MONSALVE2023	Applied Sciences (MDPI) 13(14)	Sí	JCR Q2
KITCHENHAM2005	EBSE Technical Report	No	Informe téc- nico no jour-

Totales: 45 referencias en `referencias.bib`; 40 corresponden a literatura académica/técnica citable como parte del mínimo de 30 (excluye los 5 estándares/RFC, contados aparte); de esas 40, **22 se clasifican como “alto impacto”** según el criterio estricto de la guía — cumpliendo el mínimo de 20 exigido.